Designing and Implementing a Mini Project on

GlucoGuard

Diabetes Risk Prediction & Machine-Learning Model Evaluation Platform
using Spring Boot, FastAPI and scikit-learn

Submitted to the
Savitribai Phule Pune University
In partial fulfilment for the award of the Degree of
Bachelor of Engineering
in
Information Technology

by

Kripal S Bisht (Roll No. 8329)
Himanshu Kumar (Roll No. 8322)
Kailash Bera (Roll No. 8324)

Under the guidance of

Prof. Sweta Wankhade



Department of Information Technology
Army Institute of Technology
Dighi Hills, Pune - 411 015

2026-2027



CERTIFICATE

This is to certify that **Kripal S Bisht (Roll No. 8329)**, **Himanshu Kumar (Roll No. 8322)** and **Kailash Bera (Roll No. 8324)** of the Department of Information Technology, Div B, Army Institute of Technology, Pune, have undertaken the mini project titled **"GlucoGuard: Diabetes Risk Prediction & Machine-Learning Model Evaluation Platform"** under the guidance of Prof. Sweta Wankhade. The work is submitted in partial fulfillment of the requirements for the Bachelor of Engineering degree in Information Technology for the academic year 2026–2027, as prescribed by Savitribai Phule Pune University.

The project integrates a web interface, Java Spring Boot application layer, H2 database, and Python machine-learning service for educational experimentation with binary classification, model evaluation, and saved-model inference.

Prof. Sweta Wankhade	**Dr. Sangeeta Jadhav**
Project Guide	Head of the Department
Department of Information Technology	Department of Information Technology

Place: Pune
Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to Prof. Sweta Wankhade for her guidance, feedback, and encouragement throughout the planning and development of this mini project. Her direction helped us connect machine-learning concepts with a practical web application and divide the work into manageable engineering layers.

We are thankful to Dr. Sangeeta Jadhav, Head of the Department of Information Technology, and to the teaching and technical staff of the department. We thank the Department of Information Technology and Army Institute of Technology, Pune, for the academic environment and resources that support hands-on learning. We also acknowledge the open-source communities behind Python, pandas, scikit-learn, FastAPI, Java, Spring Boot, Spring Data JPA, H2, and Chart.js.

The team recognises that responsible machine-learning work includes clear validation, reproducible experiments, careful interpretation of metrics, and honest communication of limitations.

Kripal S Bisht (8329)
Himanshu Kumar (8322)
Kailash Bera (8324)

INDEX

Sr. No.	Content	Page No.
1.	**Introduction**	**1**
	1.1 Background and Problem Statement	1
	1.2 About the Project	1
	1.3 Objectives	2
	1.4 Scope and Intended Users	2
	1.5 Tools and Technology Used	2
2.	**Feasibility Study**	**4**
	2.1 Feasibility Analysis	4
	2.2 Technical Feasibility	4
	2.3 Economic Feasibility	4
	2.4 Operational Feasibility	4
	2.5 Constraints	5
	2.6 Working of the Present System	5
	2.7 Drawbacks of the Present System	5
	2.8 Functional Requirements	5
	2.9 Non-Functional Requirements	6
3.	**Proposed System**	**7**
	3.1 Proposed System Overview	7
	3.2 Key Features and Solutions Provided	9
	3.3 Role of the User	10
	3.4 Role of the Developer	10
	3.5 Justification and Need for the System	10
4.	**Design**	**11**
	4.1 High-Level Component Architecture	11
	4.2 Responsibility Boundaries	11
	4.3 Training Pipeline and Leakage Prevention	12
	4.4 Request and Persistence Flow	13
	4.5 Dataset Description and Characteristics	13
	4.6 Feature Schema (Model Inputs)	14
	4.7 Data Validation Checklist	14
	4.8 Binary Classification Fundamentals	15
	4.9 Logistic Regression	15
	4.10 Decision Tree	15

Sr. No.	Content	Page No.
	4.11 Random Forest	16
	4.12 Comparing Candidate Algorithms	16
	4.13 Preprocessing Pipeline	17
	4.14 Hyperparameter Tuning and Cross-Validation	17
	4.15 Reproducibility	17
	4.16 Database Entities and Relationships	18
	4.17 REST API Surface	18
5.	**Implementation**	**19**
	5.1 Software Environment and Dependencies	19
	5.2 Project Structure and Running the Services	19
	5.3 Important Code Modules	21
	5.4 Job Lifecycle and Error Handling	25
	5.5 Evaluation Metrics	25
	5.6 Confusion Matrix Interpretation	26
	5.7 Results of Training Job #5	26
	5.8 Model Interpretation and Feature Importance	31
	5.9 Comparison with an Eight-Feature Training Run	32
	5.10 Summary of Recorded Training Jobs	34
	5.11 Test Strategy	35
6.	**Snapshots**	**36**
	6.1 Dashboard	36
	6.2 Results Page	37
	6.3 Make a Prediction Page	37
	6.4 History Page	38
	6.5 Health Information Section and Footer	40
7.	**Conclusion and Future Scope**	**42**
	7.1 Conclusion	42
	7.2 Limitations and Responsible Use	42
	7.3 Future Scope	42
8.	**References**	**44**
A	**Appendix A: Glossary**	**45**

List of Figures

Figure No.	Title	Page No.
Figure 3.1	GlucoGuard Complete System Architecture	8
Figure 4.1	End-to-End Application Components	11
Figure 4.2	Safe Model Development Lifecycle	12
Figure 4.3	Asynchronous Training-Job Flow	13
Figure 4.4	Random Forest Ensemble Voting	16
Figure 4.5	Preprocessing Inside the Model Pipeline	17
Figure 4.6	Database Entity Relationship	18
Figure 5.1	Project Structure in IntelliJ IDEA with the Python ML Service Running	20
Figure 5.2	State Transitions for a Training Job	25
Figure 5.3	Confusion Matrix: What Each Outcome Means	26
Figure 5.4	Results Page: Summary of Training Job #5 and Model Chosen by Cross-Validation	27
Figure 5.5	Model Comparison Table with Cross-Validation and Held-Out Test Scores (Job #5)	29
Figure 5.6	Test-Set Scores by Model, ROC Curves and Confusion Matrices (Job #5)	30
Figure 5.7	Logistic Regression Coefficients and Data Split Details (Job #5)	31
Figure 5.8	Random Forest Impurity-Based Feature Importance (Job #5)	32
Figure 5.9	Confusion Matrices and Logistic Regression Coefficients of an Eight-Feature Training Run	33
Figure 5.10	Decision Tree Feature Importance of the Eight-Feature Training Run	34
Figure 6.1	Dashboard with System Status and Usage Counters	36
Figure 6.2	Dashboard: Recent Training Jobs and "How It Works" Steps	37
Figure 6.3	Make a Prediction Page for Saved Model #13 (Logistic Regression, Job #5)	38
Figure 6.4	History Page: Training Jobs Stored in the Database	39
Figure 6.5	History Page: Recent Predictions with Predicted Class and Probability	40
Figure 6.6	"Did You Know?" Health Information Section and Footer	41

Note: Figure, table and listing numbers are written as a.b, where a is the chapter number and b is the item number within that chapter.

List of Tables

Table No.	Title	Page No.
Table 1.1	Project at a Glance	2
Table 2.1	Feasibility Assessment	4
Table 2.2	Functional Requirements	5
Table 4.1	Responsibility Boundaries of Each Component	11
Table 4.2	Profile of the Synthetic Demonstration Dataset	14
Table 4.3	Demonstration Dataset and Feature Schema	14
Table 4.4	Strengths and Limitations of Logistic Regression	15
Table 4.5	Comparison of the Candidate Algorithms	16
Table 4.6	Public REST API Surface of the Spring Boot Application	18
Table 5.1	Technology Stack and Responsibilities	19
Table 5.2	Evaluation Metrics and Their Interpretation	25
Table 5.3	Worked Example of a Confusion Matrix (Illustrative Numbers)	26
Table 5.4	Configuration of Training Job #5	27
Table 5.5	Model Comparison for Training Job #5 (best value per row in bold)	28
Table 5.6	Confusion Matrix Counts on the Held-Out Test Set (Job #5)	30
Table 5.7	Test-Set Metrics of the Eight-Feature Run, Calculated from Its Confusion-Matrix Counts	33
Table 5.8	Training Jobs Recorded in the Application Database	34
Table 5.9	Test Strategy	35
Table A.1	Glossary of Terms	45

List of Code Listings

Listing No.	Title	Page No.
Listing 5.1	Spring Boot Entry Point (DiabetesRiskApplication.java)	20
Listing 5.2	Starting the Python ML Service	21
Listing 5.3	Split First; Keep Test Data Out of Tuning	21
Listing 5.4	Keep Preprocessing Attached to the Estimator	22
Listing 5.5	Tune Only on Training Data; Evaluate Once on Test Data	23
Listing 5.6	Serialize Model and Metadata	24
Listing 5.7	Return a Job Resource, Not a Long-Running Response	24

Acronyms

Acronym	Expansion
AI / ML	Artificial Intelligence / Machine Learning
AIT	Army Institute of Technology
API / REST	Application Programming Interface / Representational State Transfer
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
CV	Cross-Validation
DT / LR / RF	Decision Tree / Logistic Regression / Random Forest
DTO	Data Transfer Object
EDA	Exploratory Data Analysis
F1	Harmonic mean of precision and recall
H2 / JPA	Java SQL database / Java Persistence API
HTTP 202	Request accepted for asynchronous processing
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
SPPU	Savitribai Phule Pune University
TP, TN, FP, FN	True Positive, True Negative, False Positive, False Negative
UI	User Interface

Abstract

GlucoGuard is an educational full-stack application for experimenting with diabetes-related binary classification. Users upload a labelled CSV, inspect its structure, choose a target and input features, configure a train/test split, select algorithms and evaluation metrics, and initiate training through a guided wizard. The application compares Logistic Regression and Random Forest, with Decision Tree available as an additional baseline. Results include test metrics, confusion matrices, ROC curves, and feature signals where supported by the fitted estimator.

The frontend uses HTML, CSS, and vanilla JavaScript. Java Spring Boot exposes REST endpoints, validates requests, manages asynchronous training jobs, persists metadata in H2 through Spring Data JPA, and forwards controlled requests to a Python FastAPI service. Python uses pandas and scikit-learn to validate data, build preprocessing/model pipelines, tune using cross-validation on training data only, evaluate on held-out test data, and save fitted pipelines with joblib for later predictions.

In the recorded run (Job #5) on an 800-row synthetic file, 730 rows remained after duplicates were removed and were split 70/30 with stratification. Logistic Regression was selected by 5-fold cross-validation (mean accuracy 0.696) and achieved a held-out ROC-AUC of 0.817, accuracy of 0.750, precision of 0.807 and recall of 0.511 on 220 test rows.

The bundled demonstration dataset is synthetic. Scores from it demonstrate software workflow only and are not clinical evidence. GlucoGuard is not a medical device and does not diagnose diabetes.

Keywords: Binary Classification, Logistic Regression, Random Forest, Decision Tree, Scikit-learn, FastAPI, Spring Boot, H2, REST API, Model Evaluation.

Chapter 1
Introduction

1.1 Background and Problem Statement

Diabetes is a long-term condition in which blood glucose stays too high, either because the body makes too little insulin or because it cannot use insulin effectively. In type 1 diabetes the body produces little or no insulin, whereas in type 2, the most common form, the body does not use insulin well. Type 2 symptoms often develop slowly over several years and some people notice none, which is why knowing the risk factors and getting checked matters. Over time, high blood glucose can damage blood vessels and nerves and affect the heart, eyes, kidneys and feet. Diagnosis is made by clinical blood tests such as A1C or fasting plasma glucose, usually confirmed by a second result.

Diabetes is a major public-health concern, and risk assessment can involve multiple interacting factors. A machine-learning classifier can learn patterns from labelled examples and estimate which class a new record resembles. Such an estimate depends on dataset quality, measurement practices, validation design, and intended use. GlucoGuard makes the modelling workflow visible and reproducible rather than presenting a prediction as a diagnosis.

The practical challenge addressed by this project is not simply training a classifier. A useful learning platform must also validate uploaded data, make modelling choices explicit, avoid leakage between training and testing, compare multiple metrics, preserve the complete preprocessing pipeline, and communicate the result with appropriate caution.

1.2 About the Project

GlucoGuard is an AI/ML mini project that turns a tabular classification experiment into a complete, working web application. The running application is titled *Diabetes Risk Prediction* and is organised around five pages: Dashboard, Train Model, Results, Predict and History. Its workflow follows five steps, in this order:

1. **Upload a CSV:** check the preview, column types and missing values.
2. **Pick the answer column:** choose the yes/no target and the input features.
3. **Split and tune:** hold out a test set first, then tune with cross-validation on the training data only.
4. **Compare models:** see cross-validation and test scores side by side.
5. **Try a prediction:** enter values for one record and see the saved model's output.

Table 1.1 summarises the project at a glance.

Aspect	Description
Primary purpose	Demonstrate a transparent end-to-end tabular classification workflow.
Frontend	HTML, CSS, vanilla JavaScript and Chart.js.
Application backend	Java, Spring Boot 3, REST controllers and DTO validation.
Persistence	Spring Data JPA with H2 for jobs, models and history metadata.
ML service	Python, FastAPI, pandas and scikit-learn.
Artifact storage	Local uploaded files and serialized fitted pipelines using joblib.
Important boundary	Educational prototype only; synthetic data is not clinical evidence.

Table 1.1: Project at a Glance

1.3 Objectives

1. Build a responsive interface for dataset upload, configuration, training, results, prediction, and history.
2. Inspect CSV structure, detect data-quality problems, and help prevent identifier or target leakage.
3. Compare Logistic Regression and Random Forest, with Decision Tree as an optional baseline.
4. Save complete preprocessing-and-model pipelines so inference matches training.
5. Separate cross-validation scores from held-out test metrics and show complementary evaluation views.
6. Persist job/model metadata while avoiding unnecessary storage of sensitive prediction inputs.

1.4 Scope and Intended Users

The intended users are students, demonstrators, and developers learning how to package machine-learning experiments into a web application. Users upload a labelled CSV and explicitly choose the target, positive class, features, split ratio, metric, and tuning level. The prototype does not connect to hospital records, recommend treatment, or establish a clinical diagnosis.

1.5 Tools and Technology Used

The complete project stack consists of the following open-source and freely available technologies:

- **Languages:** Python (machine-learning service), Java (application backend) and JavaScript (browser interface).

- **Machine learning:** pandas for data handling, scikit-learn (version 1.9.1 in the recorded runs) for preprocessing, models, tuning and metrics, and joblib for saving fitted pipelines.
- **ML service:** FastAPI served by Uvicorn on port 8000, running inside a Python virtual environment.
- **Application backend:** Spring Boot 3 with REST controllers, DTO validation, Spring Data JPA and the H2 database, served on port 8080.
- **Frontend:** HTML5, CSS3, vanilla JavaScript and Chart.js, delivered as static files by Spring Boot.
- **Development environment:** IntelliJ IDEA with an integrated terminal on Windows (PowerShell).
- **Dataset:** the bundled synthetic file *synthetic_diabetes_demo.csv*, stored with the application's static sample files.

Chapter 2
Feasibility Study

2.1 Feasibility Analysis

Before development began, the project was assessed for whether it could be completed with the available time, skills and resources. The assessment covered the technology required, the cost of tools, the effort needed by end users, and the constraints of running everything locally. Table 2.1 summarises the result, and the following sections expand on each dimension.

Dimension	**Assessment**
Technical	Python/scikit-learn support tabular classification; Spring Boot handles APIs; H2 simplifies local persistence.
Economic	The proposed stack is open-source and can run locally without a paid model API.
Operational	A guided wizard, validation messages, status polling and charts reduce command-line work.
Constraints	Model quality depends on data; local files require backups; both services must be running.

Table 2.1: Feasibility Assessment

2.2 Technical Feasibility

Python and scikit-learn provide mature, well-documented implementations of every algorithm, preprocessing step and metric the project needs. Spring Boot provides the REST layer, validation and asynchronous job handling, and H2 removes the need to install a separate database server. The datasets are small tabular files, so training runs on an ordinary laptop CPU: in the recorded Job #5, Logistic Regression trained in 1.4 s, Decision Tree in 1.5 s and Random Forest in 27.8 s including grid search. The project is therefore technically feasible.

2.3 Economic Feasibility

Python, pandas, scikit-learn, FastAPI, Java, Spring Boot, H2 and Chart.js are all free and open-source. The application runs entirely on a local machine and does not call any paid model API or cloud service, so the project required no spending and is economically feasible.

2.4 Operational Feasibility

A guided wizard, validation messages, status polling and charts reduce command-line work. A System Status panel on the Dashboard shows whether the web server, database and ML service are online, and every page carries an educational disclaimer. Users do not need programming knowledge to train or compare models.

2.5 Constraints

Model quality depends on data; local files require backups; and both services, the Spring Boot application and the Python ML service, must be running for training and prediction to work. Because the bundled dataset is synthetic, scores obtained from it demonstrate the software workflow only.

2.6 Working of the Present System

In a typical student workflow, a classification experiment lives in a notebook or a set of scripts. The CSV is loaded by hand, preprocessing such as scaling is often fitted on the full dataset, a single model is trained, and one accuracy figure is printed. The fitted model, if saved at all, is usually saved without its preprocessing steps, and there is no record of which configuration produced which result.

2.7 Drawbacks of the Present System

- **Data leakage:** fitting scalers or imputers before the split lets test information influence training and inflates scores.
- **Single-metric reporting:** accuracy alone hides the balance between missed positives and false alarms.
- **Inconsistent inference:** a classifier saved without its preprocessing pipeline can receive differently scaled or encoded inputs at prediction time.
- **No reproducibility or history:** seeds, splits and settings are rarely recorded, so results cannot be compared or repeated.
- **Limited accessibility:** users without programming knowledge cannot explore how modelling choices change the outcome.

2.8 Functional Requirements

ID	Requirement	Expected behaviour
FR-01	Upload and inspect	Accept an allowed CSV, profile columns, types, missing values and class balance.
FR-02	Configure experiment	Select target, positive class, features, split ratio, metric and candidate models.
FR-03	Train asynchronously	Return a job identifier quickly; expose queued/running/completed/ failed states.
FR-04	Compare models	Show cross-validation and held-out metrics without mixing the two.
FR-05	Save and predict	Persist model metadata and reload the fitted pipeline for schema-matched inference.
FR-06	View history	List prior jobs and models; return clear errors for invalid requests.

Table 2.2: Functional Requirements

2.9 Non-Functional Requirements

- **Reliability:** validate inputs at both the API boundary and the ML service boundary.
- **Maintainability:** keep UI, API orchestration, persistence, and ML responsibilities separate.
- **Reproducibility:** use explicit random seeds and save configuration and model metadata.
- **Usability:** show progress, readable charts, descriptive validation messages, and consistent navigation.
- **Privacy:** do not persist raw prediction input values in history unless there is a clear, protected requirement.

Chapter 3
Proposed System

3.1 Proposed System Overview

The proposed GlucoGuard platform is organised into five integrated parts, shown together in Figure 3.1:

1. **Frontend (User Interface):** HTML, CSS and JavaScript pages served as static files by Spring Boot, for uploading datasets, configuring training, viewing results, making predictions and reviewing history.
2. **Spring Boot Backend (Application Layer, port 8080):** REST controllers, a service layer for training jobs, predictions and calls to the ML service, and a Spring Data JPA persistence layer.
3. **Python ML Service (FastAPI, port 8000):** an ML processing pipeline of data validation, preprocessing, model training, model evaluation, model persistence and prediction.
4. **Data Storage:** uploaded CSV datasets and trained model artifacts (joblib pipelines with JSON metadata) kept as local files.
5. **H2 Database:** application metadata for training jobs, trained models and prediction history.

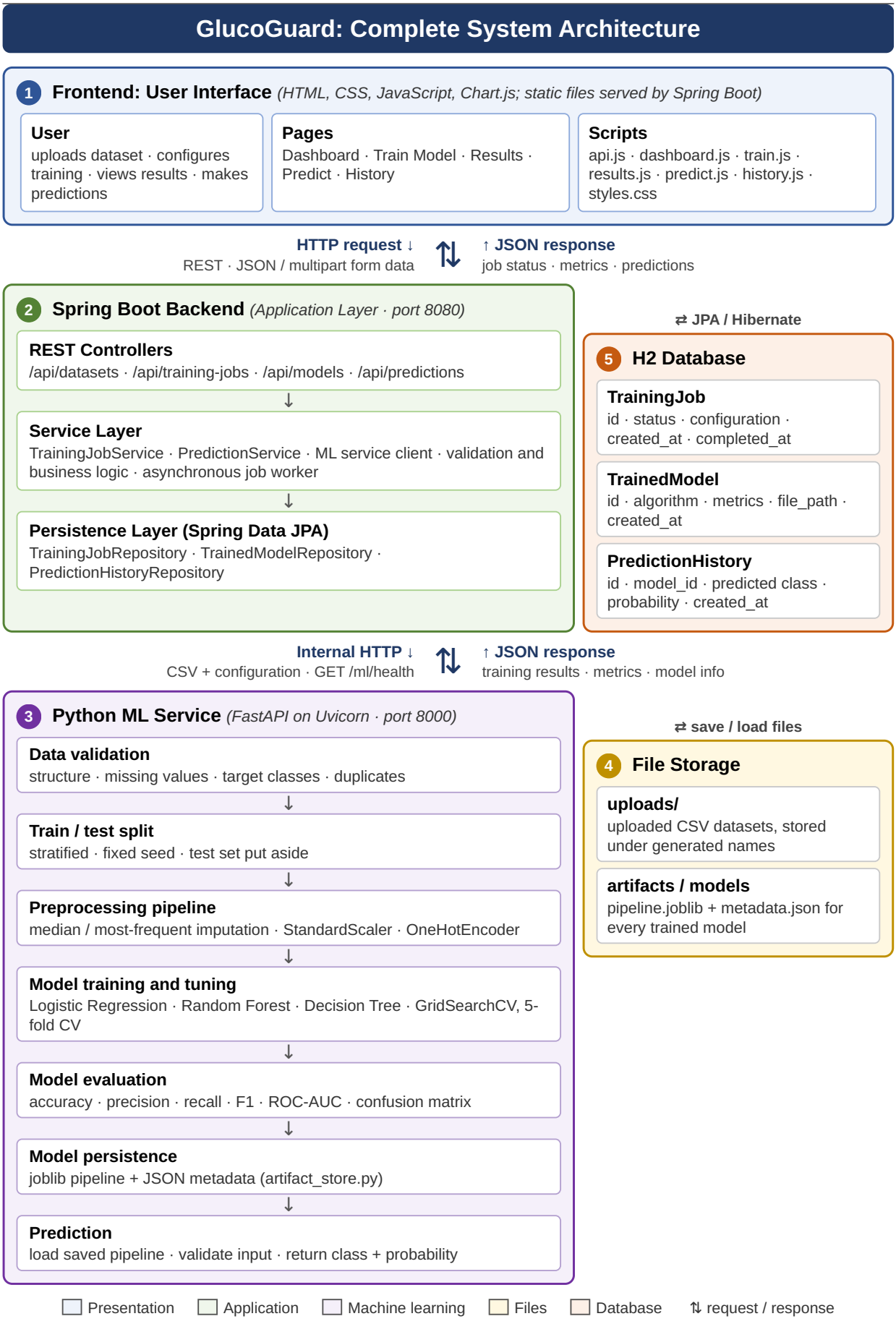


Figure 3.1: GlucoGuard Complete System Architecture

Figure 3.1 is read from top to bottom. Requests travel downwards from the browser to Spring Boot and, for machine-learning work, on to the Python service; responses travel back up as JSON. The browser never calls the ML service directly. Two flows run through this architecture:

- **Training flow:** the browser uploads the CSV and configuration; Spring Boot validates the request, stores the file and creates a job with status QUEUED; a worker sends the dataset and configuration to the ML service, which validates the data, splits it, tunes and evaluates each candidate model and saves the fitted pipelines; Spring Boot then saves the metrics and model metadata in H2, marks the job COMPLETED, and the Results page fetches and displays them.
- **Prediction flow:** the user fills in the Predict form for a chosen model; Spring Boot validates the request and calls the ML service, which loads the saved pipeline, checks the input against the model's features and returns the predicted class and probability; Spring Boot records the class and probability (not the inputs) in the prediction history and returns the result to the page.

3.2 Key Features and Solutions Provided

- **Guided training wizard:** the user uploads a CSV, reviews its profile, and chooses the target, positive class, features, split, metric and tuning level in plain language.
- **Leakage-safe evaluation:** the test set is put aside before any tuning, and the model is chosen by cross-validation on training data only. The Results page states explicitly that test-set results were not used to choose the model.
- **Multi-model comparison:** Logistic Regression, Random Forest and Decision Tree are compared on the same split with cross-validation scores, test metrics, ROC curves and confusion matrices.
- **Model interpretation:** coefficients for Logistic Regression and impurity-based importance for the tree models, each with a caution that they are not causation or clinical significance.
- **Schema-matched prediction:** the prediction form is built from the exact features a saved model was trained on, shows the training range and median for each input, and warns when a value lies outside that range.
- **Exportable reports and history:** every job can be downloaded as a CSV or JSON report, and the History page lists all jobs and predictions.
- **Privacy by design:** prediction inputs are never stored; only the predicted class and probability are kept.
- **Clear labelling:** synthetic datasets carry a "Synthetic" badge and a demonstration-only warning on every result.

3.3 Role of the User

1. **Check status:** open the Dashboard and confirm that the web server, database and ML service are online.
2. **Train:** upload a labelled CSV, choose the target and features, select the models, metric and tuning level, and start training.
3. **Review:** open the Results page to compare cross-validation and test scores, ROC curves, confusion matrices and feature signals, and download the report.
4. **Predict:** choose a saved model, enter values for one record and read the predicted class and probability, together with the disclaimer.
5. **Track:** use the History page to revisit earlier jobs and predictions.

3.4 Role of the Developer

1. **Maintain services:** keep the Spring Boot application and the Python ML service configured, running and reachable from each other.
2. **Maintain the ML pipeline:** update validation rules, parameter grids and supported algorithms in the Python service, and record library versions.
3. **Maintain the API and data:** manage REST endpoints, DTO validation, JPA entities and the storage locations for uploads and artifacts.
4. **Test and verify:** run data-validation, ML, integration, persistence, UI and security checks (Section 5.11) after each change.

3.5 Justification and Need for the System

Most introductory machine-learning work stops at a notebook and a single accuracy number. GlucoGuard shows the full discipline around a model: validating data, preventing leakage, comparing several algorithms with complementary metrics, saving reproducible artifacts and communicating limitations. Packaging this as a web application makes the workflow accessible to users who do not write code, and makes the separation between training data, validation and held-out testing visible on every screen. The system is intended for education and demonstration; it is not a diagnostic tool.

Chapter 4
Design

4.1 High-Level Component Architecture

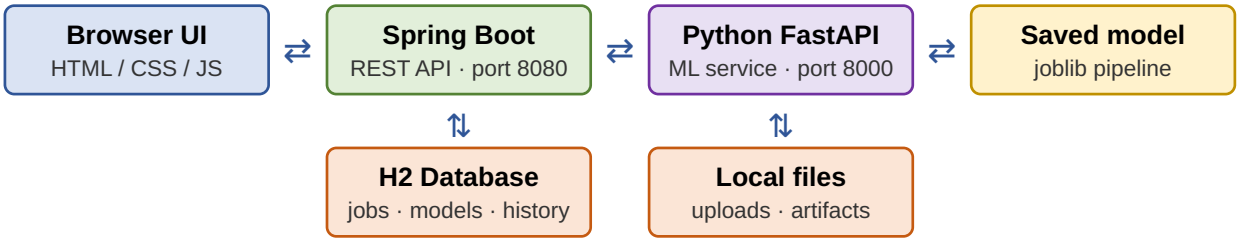


Figure 4.1: End-to-End Application Components

The browser communicates with Spring Boot on port 8080. Spring Boot serves pages and owns the public REST API, validates requests, stores uploaded CSVs, persists job/model/history metadata in H2, and calls FastAPI on port 8000. Python owns CSV profiling, model training, evaluation, serialization, and inference. The UI should not call the ML service directly; this keeps validation and orchestration behind one application boundary. The Dashboard confirms this design to the user: the browser talks only to the web server, which calls the ML service internally.

4.2 Responsibility Boundaries

Component	Owns	Should not own
Browser UI	Wizard state, form validation feedback, progress, charts and user interactions.	Model fitting, database credentials or direct artifact file access.
Spring Boot	Public endpoints, request validation, job lifecycle, metadata persistence and error mapping.	Python estimator internals or direct editing of fitted model objects.
FastAPI ML service	Data profiling, preprocessing, cross-validation, evaluation, prediction and artifact serialization.	Public UI sessions or primary application metadata.
H2 database	Job, model and prediction-history metadata.	Large raw datasets or serialized model binaries.
Local file storage	Uploaded CSVs and fitted model artifacts.	Unrestricted public access to user-supplied files.

Table 4.1: Responsibility Boundaries of Each Component

4.3 Training Pipeline and Leakage Prevention

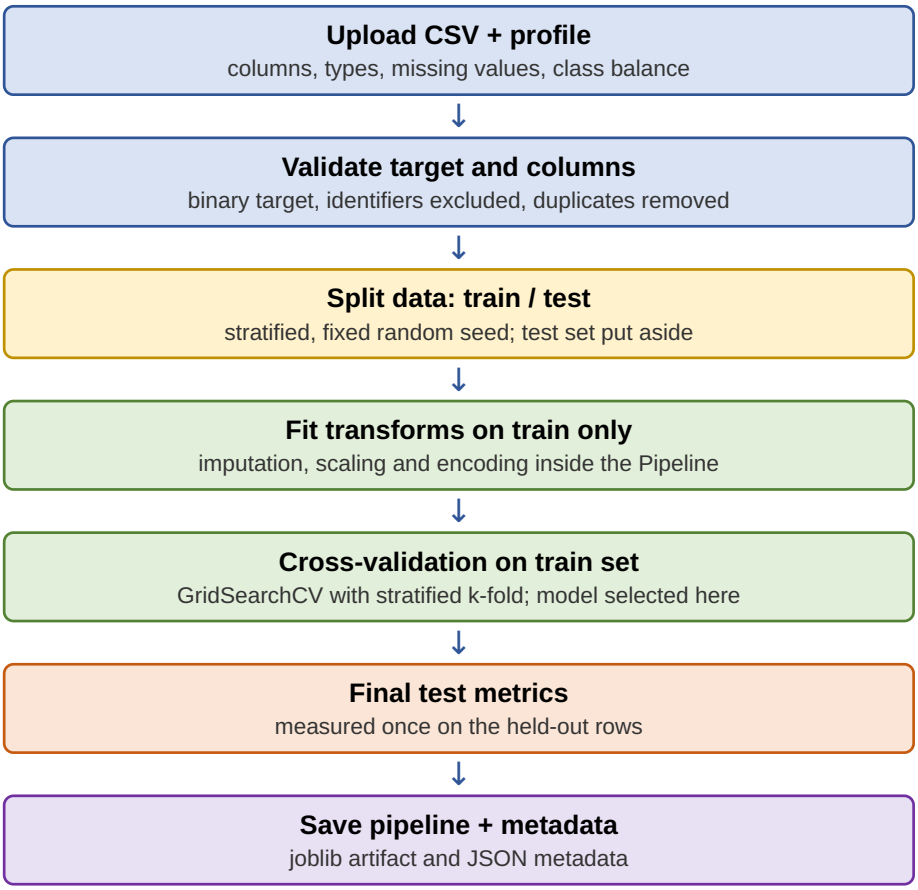


Figure 4.2: Safe Model Development Lifecycle

The split occurs before fitting transformations. Imputation values, scaling parameters, and category vocabularies are learned from training rows only. Cross-validation and hyperparameter search use training data; the held-out test set is reserved for final comparison. This reduces optimistic estimates caused by data leakage. If preprocessing is fitted on the full dataset before the split, information from the test set can influence the training process and make evaluation look better than it really is.

4.4 Request and Persistence Flow

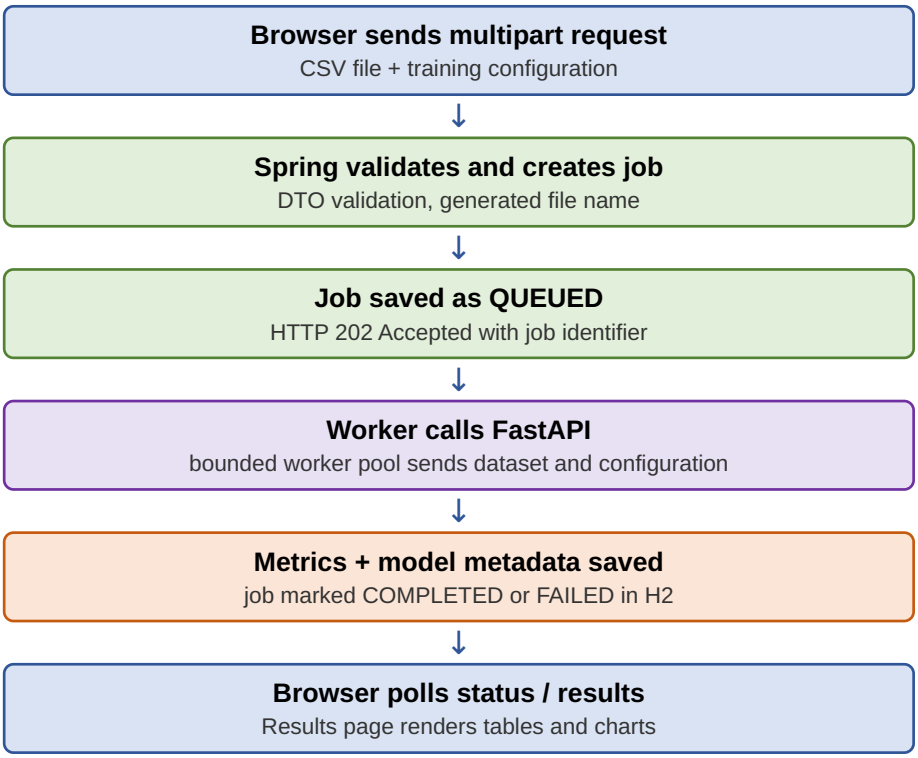


Figure 4.3: Asynchronous Training-Job Flow

The training request should return HTTP 202 Accepted with a job identifier. This means the request has been accepted for processing, not that training is already complete. The browser can poll a status endpoint or use a future push-based mechanism. A bounded worker pool prevents an unbounded number of expensive training tasks from overwhelming the server.

4.5 Dataset Description and Characteristics

The application ships with a synthetic demonstration dataset, *synthetic_diabetes_demo.csv*. The project specification describes a synthetic dataset of roughly 500 rows and 10 columns; the actual upload profile reported by the application is used below for the row count, duplicates and class balance. Table 4.2 summarises the profile from training Job #5, which used Glucose and Age as features.

Attribute	Value
File	synthetic_diabetes_demo.csv (synthetic)
Rows in file	800
Duplicate rows removed (Job #5)	70
Rows without a label removed	0
Rows used (Job #5)	730
Class 0 (negative)	431 rows (59.0%): 301 training, 130 test
Class 1 (positive class)	299 rows (41.0%): 209 training, 90 test
Target column	Diabetes (positive class = 1)
Identifier column	PatientID (excluded by default)

Table 4.2: Profile of the Synthetic Demonstration Dataset

Do not claim that this schema or row count represents a real clinical population. The class split shows a moderate imbalance, which is why the split is stratified: both partitions keep roughly the same proportion of positive rows.

4.6 Feature Schema (Model Inputs)

Column	Role / type	Handling and notes
PatientID	Identifier	Exclude by default; identifiers often have no predictive meaning.
Pregnancies	Numeric feature	Median imputation; scaling for Logistic Regression.
Glucose	Numeric feature	Synthetic demonstration values only; not a clinical threshold.
BloodPressure	Numeric feature	Numeric imputation and scaling.
BMI	Numeric feature	Numeric imputation and scaling.
DiabetesPedigreeFunction	Numeric feature	Numeric predictor in the demonstration schema.
Age	Numeric feature	Numeric imputation and scaling.
PhysicalActivity	Categorical feature	Most-frequent imputation and one-hot encoding (Low, Moderate, High).
FamilyHistory	Categorical feature	Most-frequent imputation and one-hot encoding.
Diabetes	Binary target	Positive class must be explicitly confirmed.

Table 4.3: Demonstration Dataset and Feature Schema

4.7 Data Validation Checklist

- Reject files that are not CSV or exceed the configured size limit.

- Check for blank or duplicate column names, empty files, and insufficient rows.
- Confirm the selected target exists and contains exactly two usable classes for binary classification.
- Detect constant columns, identifiers, extreme missingness, and unsupported high-cardinality text.
- Check class counts before stratified splitting; report why a split cannot be performed.
- Treat user-supplied filenames as untrusted and generate server-side storage names.

4.8 Binary Classification Fundamentals

Binary classification assigns an input record to one of two classes, conventionally labelled 0 and 1. The feature vector is written as $x = (x_1, x_2, \ldots, x_n)$. A model may output a class directly, or a probability estimate for the positive class. A threshold converts that probability into a class label. A threshold of 0.5 is common for demonstration, but it is not automatically the right threshold for every application.

4.9 Logistic Regression

Despite its name, Logistic Regression is a classification algorithm. It calculates a weighted sum of features and maps the result to a value between 0 and 1 using the sigmoid function. During training, it learns weights by minimising a regularised log-loss objective. It is a useful baseline because it is efficient and its coefficients can provide directional insight after suitable encoding and scaling.

$$p(y = 1 \mid x) = 1 / (1 + \exp(-(w \cdot x + b)))$$

Strengths	**Limitations**
Fast to train; probability output; interpretable baseline.	Linear decision boundary in transformed feature space; nonlinear interactions may need feature engineering.
Scaling numeric inputs often improves optimisation.	Coefficients should not be interpreted as causation.

Table 4.4: Strengths and Limitations of Logistic Regression

4.10 Decision Tree

A Decision Tree recursively partitions the feature space using rules such as "Glucose ≤ threshold?" or "FamilyHistory = Yes?". At each node, a split is selected to reduce impurity according to a criterion such as Gini impurity or entropy. Leaf nodes output a class or class probability. Trees can model nonlinear relationships and are comparatively easy to visualise, but an unconstrained tree can memorise training examples.

$$Gini = 1 - \Sigma\, p_i^2, \text{ where } p_i \text{ is the fraction of class } i \text{ in a node.}$$

Useful controls include max_depth, min_samples_split, min_samples_leaf, and class_weight. These settings constrain tree complexity and can improve generalisation.

4.11 Random Forest

Random Forest is an ensemble of decision trees. Each tree is trained on a bootstrap sample, and each split considers a random subset of features. For classification, the forest combines votes or averages predicted probabilities. Bagging and feature randomness can reduce the variance of a single tree and often provide a strong baseline for tabular data.

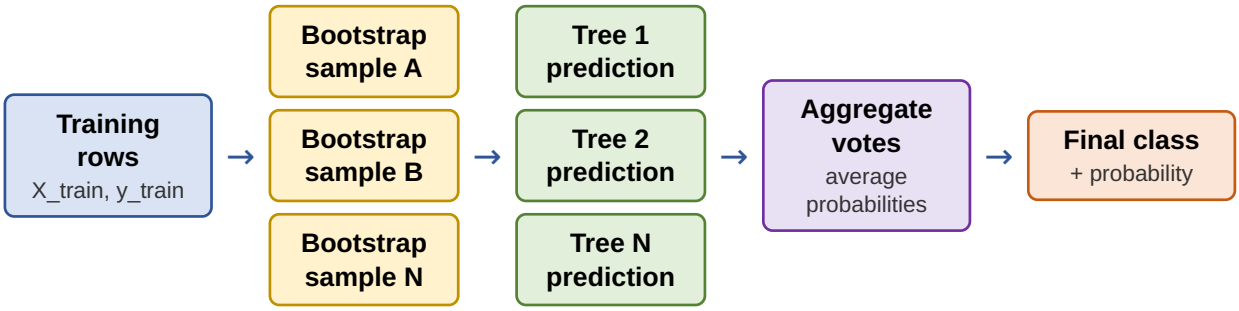


Figure 4.4: Random Forest Ensemble Voting

The forest can be configured with the number of trees (n_estimators), maximum depth, minimum leaf size, and number of features considered at each split (max_features). More trees can stabilise predictions but increase training and inference cost. Feature importance describes model reliance under a particular calculation; it does not prove that a feature causes the outcome.

4.12 Comparing Candidate Algorithms

Algorithm	Learning behaviour	Advantages	Watch-outs
Logistic Regression	Linear probability model after sigmoid transform.	Fast baseline; coefficient-based interpretation; probability output.	May miss nonlinear patterns; benefits from scaling.
Decision Tree	Rule-based recursive splits.	Readable decision rules; handles nonlinear interactions.	Can overfit without depth/ leaf constraints.
Random Forest	Ensemble of decorrelated decision trees.	Often robust on tabular data; captures nonlinear patterns.	Less transparent than one tree; more compute and memory.

Table 4.5: Comparison of the Candidate Algorithms

4.13 Preprocessing Pipeline

Numeric features commonly use median imputation, followed by StandardScaler for Logistic Regression. Categorical features use most-frequent imputation followed by OneHotEncoder. A scikit-learn ColumnTransformer applies the correct transformations to each column, while Pipeline binds those transformations to the estimator. Saving the complete pipeline ensures future inference uses the same learned imputation, scaling, and encoding.

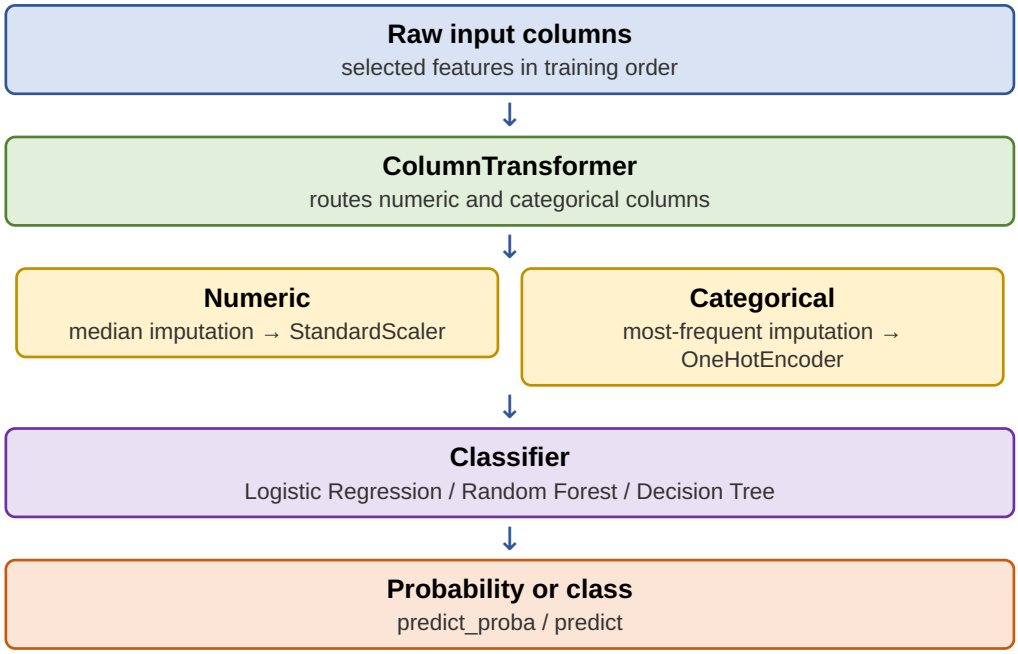


Figure 4.5: Preprocessing Inside the Model Pipeline

4.14 Hyperparameter Tuning and Cross-Validation

GridSearchCV evaluates a bounded set of parameter combinations using k-fold cross-validation. In each fold, one partition is used for validation while the remaining folds train the candidate. The selected scoring metric identifies the best configuration. The held-out test set must not be used to choose hyperparameters; otherwise, the final test score becomes biased by repeated selection.

In the application the user chooses a tuning level. At the "Balanced" level used in Job #5, 5-fold cross-validation was run over 8 settings for Logistic Regression and 12 settings each for Random Forest and Decision Tree; a "Thorough" level is also available (used in Job #2).

4.15 Reproducibility

- Record the random seed, split fraction, scoring metric, candidate models and tuning parameters.
- Store a dataset fingerprint and feature schema where appropriate.
- Save the fitted preprocessing-plus-estimator pipeline, not only the classifier.
- Separate cross-validation results from the final held-out test metrics.

- Record library versions so future runs can be compared more reliably. Each saved model in the application records its pipeline version and scikit-learn version (pipeline v1, scikit-learn 1.9.1).

4.16 Database Entities and Relationships

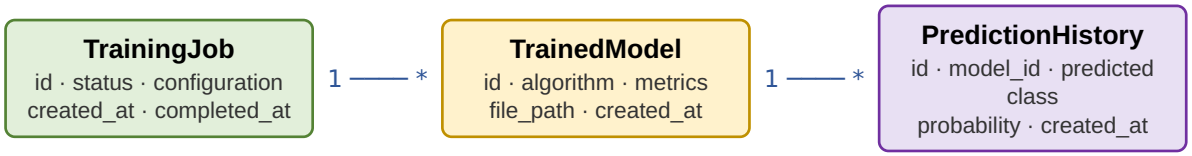


Figure 4.6: Database Entity Relationship

A completed training job can be associated with one or more model records, depending on whether each candidate is saved. Prediction history should reference the model identifier and timestamp. Avoid storing raw health-related input values in history unless the system has a clear purpose, access controls, retention rules, and user notice. In the implemented application, every candidate model is saved (three per job), and the History page confirms that prediction inputs are never stored, only the predicted class and probability.

4.17 REST API Surface

Method	Endpoint	Purpose
POST	/api/datasets/profile	Upload and profile a CSV before training.
POST	/api/training-jobs	Create an asynchronous training job.
GET	/api/training-jobs/{id}	Read job status and progress.
GET	/api/training-jobs/{id}/results	Read completed metrics and comparison.
GET	/api/models	List saved model metadata.
POST	/api/models/{id}/predict	Run prediction using the saved pipeline.
GET	/api/history	List prediction history metadata.

Table 4.6: Public REST API Surface of the Spring Boot Application

These routes describe the interface consistent with the report's architecture; the resource groups /api/datasets, /api/training-jobs, /api/models and /api/predictions are shown in the REST Controllers block of Figure 3.1. Behind this public API, the Python service exposes internal endpoints that only Spring Boot calls, such as the health check GET /ml/health, which the Dashboard's System Status panel uses (visible in the terminal log in Figure 5.1).

Chapter 5
Implementation

5.1 Software Environment and Dependencies

Layer	Technology	Responsibility
Presentation	HTML5, CSS3, vanilla JS, Chart.js	Wizard, validation, progress, charts, prediction and history.
Application backend	Java, Spring Boot 3, REST, DTOs	API boundary, validation, job lifecycle, error mapping.
Persistence	Spring Data JPA + H2	TrainingJob, TrainedModel and PredictionHistory metadata.
ML service	Python FastAPI, pandas, scikit-learn 1.9.1, Uvicorn	CSV profiling, preprocessing, tuning, evaluation and inference.
Artifact storage	Local files + joblib	Uploaded CSVs and complete fitted pipelines.
Tooling	IntelliJ IDEA, Python virtual environment, PowerShell	Development, running both services and viewing logs.

Table 5.1: Technology Stack and Responsibilities

5.2 Project Structure and Running the Services

The Java application lives in the package *edu.ait.diabetesrisk*, divided into the sub-packages **client** (HTTP client for the ML service), **config**, **controller**, **dto**, **entity**, **exception**, **repository** and **service**, with *DiabetesRiskApplication* as the entry point. The frontend is held under *resources/static*: *css/styles.css*, local fonts, and one JavaScript file per page (*api.js*, *dashboard.js*, *train.js*, *results.js*, *predict.js* and *history.js*), plus a *vendor* folder for third-party scripts and the *sample* folder with *synthetic_diabetes_demo.csv*. The Python service lives in a separate *ml-service* folder, with modules such as *artifact_store.py* and *config.py*. Figure 5.1 shows this structure in the IDE together with the ML service running in the terminal.

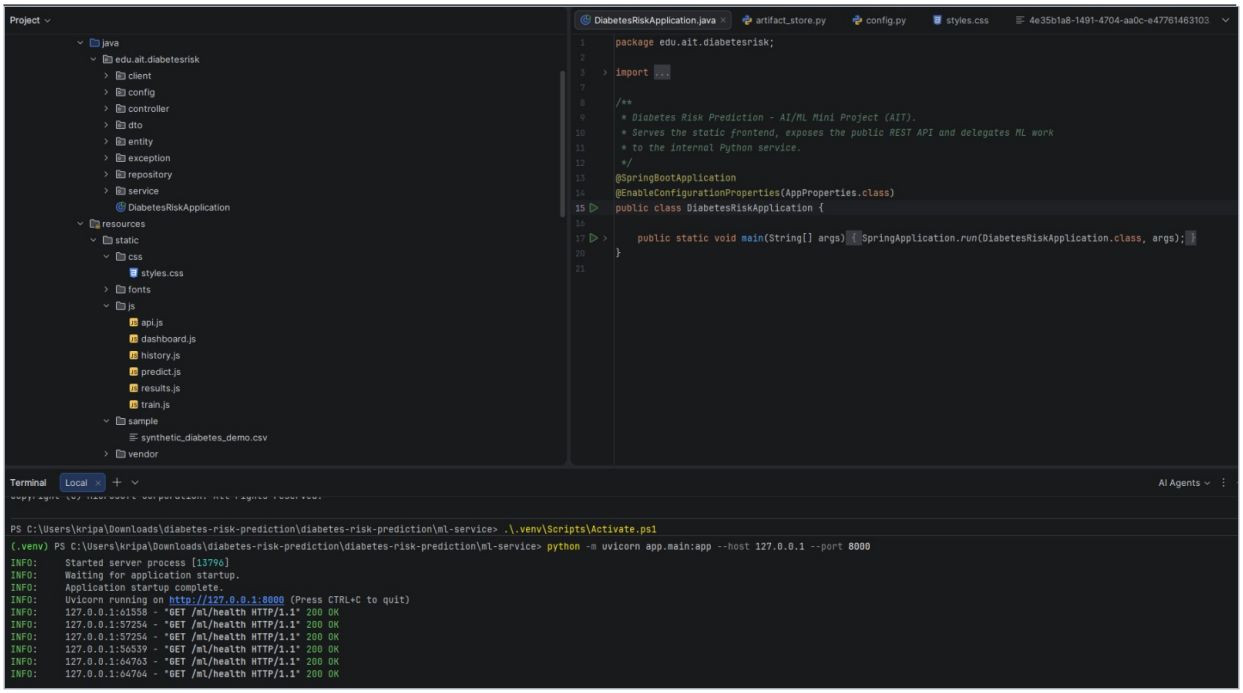


Figure 5.1: Project Structure in IntelliJ IDEA with the Python ML Service Running

The Spring Boot entry point enables typed configuration properties, which hold settings such as the ML service address:

```
package edu.ait.diabetesrisk;

/**
 * Diabetes Risk Prediction - AI/ML Mini Project (AIT).
 * Serves the static frontend, exposes the public REST API and delegates ML work
 * to the internal Python service.
 */
@SpringBootApplication
@EnableConfigurationProperties(AppProperties.class)
public class DiabetesRiskApplication {

    public static void main(String[] args) {
        SpringApplication.run(DiabetesRiskApplication.class, args);
    }
}
```

Listing 5.1: Spring Boot Entry Point (DiabetesRiskApplication.java)

The ML service is started from its own virtual environment. Once Uvicorn reports that it is running, the Dashboard's periodic health checks appear in the log as successful GET /ml/health requests:

```
PS ...\ml-service> .\.venv\Scripts\Activate.ps1
(.venv) PS ...\ml-service> python -m uvicorn app.main:app --host 127.0.0.1 --port
8000
INFO:     Started server process [13796]
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:61558 - "GET /ml/health HTTP/1.1" 200 OK
```

Listing 5.2: Starting the Python ML Service

5.3 Important Code Modules

The listings in this section show the implementation pattern used by the ML service and the Spring Boot API for each critical step. They are representative excerpts written for clarity; variable names may differ slightly from the project source files.

(a) Python: safe train/test split

```
from sklearn.model_selection import train_test_split

X = frame[feature_columns]
y = frame[target_column]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=1.0 - train_fraction,
    stratify=y,
    random_state=seed,
)

# Do not fit imputers, scalers or encoders on the full dataset.
# Fit them inside a Pipeline using X_train only.
```

Listing 5.3: Split First; Keep Test Data Out of Tuning

Why this matters: stratify=y aims to preserve the class proportions in each partition. A fixed random_state makes the split reproducible. Stratification can fail if a class has too few examples, so validate class counts before calling the function.

(b) Python: preprocessing and estimator pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_pipe, numeric_columns),
    ("categorical", categorical_pipe, categorical_columns),
])

pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("classifier", LogisticRegression(max_iter=2000)),
])
```

Listing 5.4: Keep Preprocessing Attached to the Estimator

The exact feature lists should be derived from the validated dataset profile. handle_unknown='ignore' prevents inference from failing solely because a categorical column contains a category that was not present during fitting.

(c) Python: tuning and held-out evaluation

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)

search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring=scoring_metric,
    cv=cv,
    n_jobs=-1,
    refit=True,
)
search.fit(X_train, y_train)

best_pipeline = search.best_estimator_
predicted = best_pipeline.predict(X_test)
probability = best_pipeline.predict_proba(X_test)[:, 1]

metrics = {
    "accuracy": accuracy_score(y_test, predicted),
    "precision": precision_score(y_test, predicted, zero_division=0),
    "recall": recall_score(y_test, predicted, zero_division=0),
    "f1": f1_score(y_test, predicted, zero_division=0),
    "roc_auc": roc_auc_score(y_test, probability),
    "confusion_matrix": confusion_matrix(y_test, predicted).tolist(),
}
```

Listing 5.5: Tune Only on Training Data; Evaluate Once on Test Data

Important checks: the positive class must be mapped consistently; the selected estimator must support predict_proba for the ROC-AUC calculation shown; and ROC-AUC requires both classes in the test set. In a production implementation, wrap these conditions with explicit validation and clear error messages.

(d) Python: save the full fitted pipeline

```
from pathlib import Path
import joblib
import json

artifact_dir = Path("artifacts") / model_id
artifact_dir.mkdir(parents=True, exist_ok=True)

joblib.dump(best_pipeline, artifact_dir / "pipeline.joblib")

metadata = {
    "model_id": model_id,
    "algorithm": algorithm_name,
    "target_column": target_column,
    "feature_columns": feature_columns,
    "positive_class": str(positive_class),
    "random_seed": seed,
    "metrics": metrics,
}

(artifact_dir / "metadata.json").write_text(
    json.dumps(metadata, indent=2),
    encoding="utf-8",
)
```

Listing 5.6: Serialize Model and Metadata

Only load joblib files from trusted storage. Serialized Python objects can execute code during loading; never accept an arbitrary model file uploaded by an untrusted user.

(e) Spring Boot: asynchronous training endpoint

```
@PostMapping(
    path = "/training-jobs",
    consumes = MediaType.MULTIPART_FORM_DATA_VALUE
)
public ResponseEntity<TrainingJobDto> create(
        @RequestPart("file") MultipartFile file,
        @Valid @RequestPart("config") TrainingConfigRequest config) {

    TrainingJobDto job = trainingService.create(file, config);
    URI statusUri = URI.create(
        "/api/training-jobs/" + job.id()
    );

    return ResponseEntity.accepted()
        .location(statusUri)
        .body(job);
}
```

Listing 5.7: Return a Job Resource, Not a Long-Running Response

HTTP 202 Accepted means the job has been queued, not that training has completed. The browser should poll GET /api/training-jobs/{id}. The service should enforce file-size limits, validate allowed file types, use generated filenames, and return structured error responses.

5.4 Job Lifecycle and Error Handling

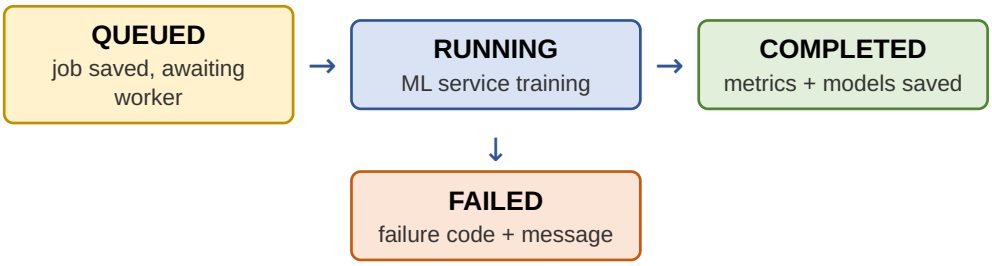


Figure 5.2: State Transitions for a Training Job

A job may transition from QUEUED to RUNNING and then to COMPLETED or FAILED. Persist timestamps and a concise failure code/message. Do not expose stack traces, file-system paths, credentials, or internal service details to the browser. Retry rules should be explicit because training may be expensive and some failures are not transient.

5.5 Evaluation Metrics

Metric	Definition	Interpretation
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Fraction of all predictions that are correct; can mislead with imbalanced classes.
Precision	TP / (TP + FP)	Of records predicted positive, how many are actually positive?
Recall / Sensitivity	TP / (TP + FN)	Of actual positives, how many did the model identify?
F1-score	2 × Precision × Recall / (Precision + Recall)	Balances precision and recall; useful when both matter.
ROC-AUC	Area under the ROC curve	Measures ranking across thresholds; it is not the same as calibration.
Confusion matrix	Counts TN, FP, FN and TP	Shows the types of mistakes, not just the overall rate.

Table 5.2: Evaluation Metrics and Their Interpretation

5.6 Confusion Matrix Interpretation

		Predicted class	
		Predicted 0	**Predicted 1**
Actual class	**Actual 0**	**TN** Correctly predicted 0	**FP** False alarm
	Actual 1	**FN** Missed positive	**TP** Correctly predicted 1

Figure 5.3: Confusion Matrix: What Each Outcome Means

False positives and false negatives represent different kinds of error. In a medical context, the costs of these errors depend on the intended use and must be assessed by qualified domain experts. This educational application does not define clinical thresholds.

Worked example using illustrative numbers

The following values are a mathematical example only and are not results from GlucoGuard. Suppose a test set contains 100 records: 40 positive and 60 negative. The model identifies 30 of the 40 positive records correctly, misses 10, correctly rejects 48 negatives, and incorrectly flags 12 negatives.

	Predicted 0	**Predicted 1**	**Actual total**
Actual 0	TN = 48	FP = 12	60
Actual 1	FN = 10	TP = 30	40
Predicted total	58	42	100

Table 5.3: Worked Example of a Confusion Matrix (Illustrative Numbers)

Accuracy = (48 + 30) / 100 = 78%. Precision = 30 / (30 + 12) ≈ 71.4%. Recall = 30 / (30 + 10) = 75%. F1 ≈ 73.2%. This example shows why a single accuracy number cannot fully describe a classifier.

5.7 Results of Training Job #5

The results below come from training Job #5, run on 30 October 2026 at 4:11 PM on the synthetic dataset. All three candidate models were trained on the same split; Table 5.4 lists the configuration as recorded by the application, and Figure 5.4 shows the Results page header for this job.

Setting	Value
Dataset	synthetic_diabetes_demo.csv (synthetic/demo)
Target / positive class	Diabetes / 1
Features used	Glucose, Age
Rows used	730 (800 in file, 70 duplicates removed)
Split	70% train / 30% test, stratified (510 / 220 rows)
Selection metric	Accuracy
Tuning	Balanced, 5-fold cross-validation (GridSearchCV)
Random seed	42
Candidate models	Logistic Regression, Random Forest, Decision Tree

Table 5.4: Configuration of Training Job #5

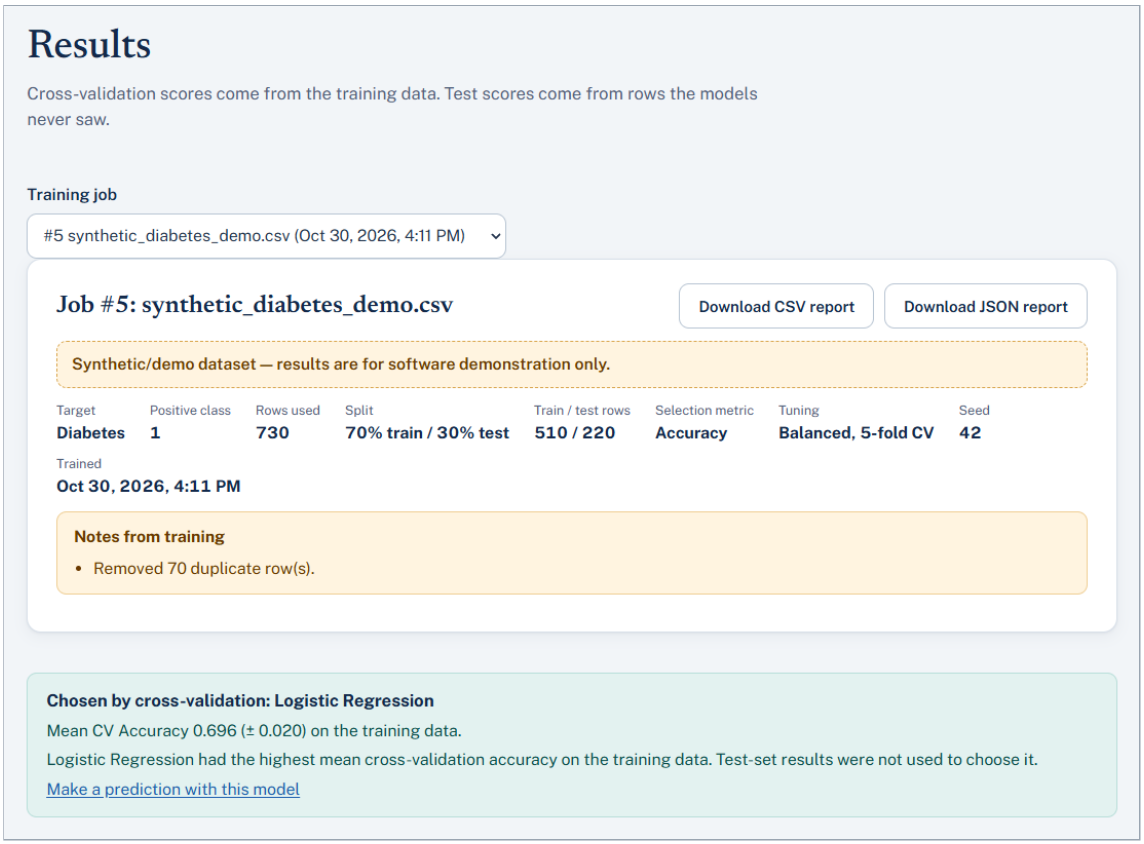
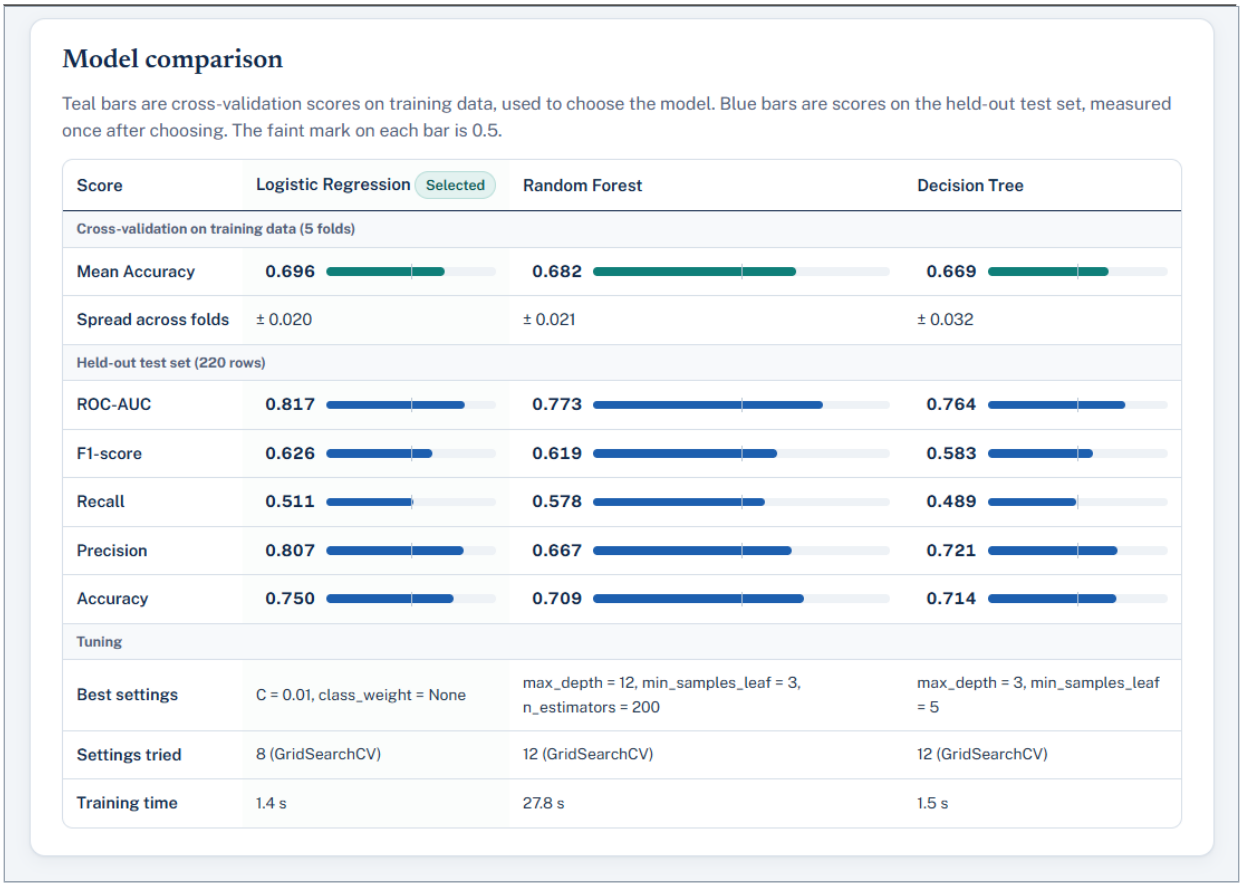


Figure 5.4: Results Page: Summary of Training Job #5 and Model Chosen by Cross-Validation

Logistic Regression had the highest mean cross-validation accuracy on the training data (0.696 ± 0.020) and was therefore selected. The test-set results were not used to choose it. Table 5.5 compares all three models, and Figure 5.5 shows the same comparison as displayed by the application.

Score	Logistic Regression (selected)	Random Forest	Decision Tree
Cross-validation on training data (5 folds)			
Mean accuracy	**0.696**	0.682	0.669
Spread across folds	± 0.020	± 0.021	± 0.032
Held-out test set (220 rows)			
ROC-AUC	**0.817**	0.773	0.764
F1-score	**0.626**	0.619	0.583
Recall	0.511	**0.578**	0.489
Precision	**0.807**	0.667	0.721
Accuracy	**0.750**	0.709	0.714
Tuning			
Best settings	C = 0.01, class_weight = None	max_depth = 12, min_samples_leaf = 3, n_estimators = 200	max_depth = 3, min_samples_leaf = 5
Settings tried (GridSearchCV)	8	12	12
Training time	1.4 s	27.8 s	1.5 s

Table 5.5: Model Comparison for Training Job #5 (best value per row in bold)



Model comparison

Teal bars are cross-validation scores on training data, used to choose the model. Blue bars are scores on the held-out test set, measured once after choosing. The faint mark on each bar is 0.5.

Score	Logistic Regression Selected	Random Forest	Decision Tree
Cross-validation on training data (5 folds)			
Mean Accuracy	0.696	0.682	0.669
Spread across folds	± 0.020	± 0.021	± 0.032
Held-out test set (220 rows)			
ROC-AUC	0.817	0.773	0.764
F1-score	0.626	0.619	0.583
Recall	0.511	0.578	0.489
Precision	0.807	0.667	0.721
Accuracy	0.750	0.709	0.714
Tuning			
Best settings	C = 0.01, class_weight = None	max_depth = 12, min_samples_leaf = 3, n_estimators = 200	max_depth = 3, min_samples_leaf = 5
Settings tried	8 (GridSearchCV)	12 (GridSearchCV)	12 (GridSearchCV)
Training time	1.4 s	27.8 s	1.5 s

Figure 5.5: Model Comparison Table with Cross-Validation and Held-Out Test Scores (Job #5)

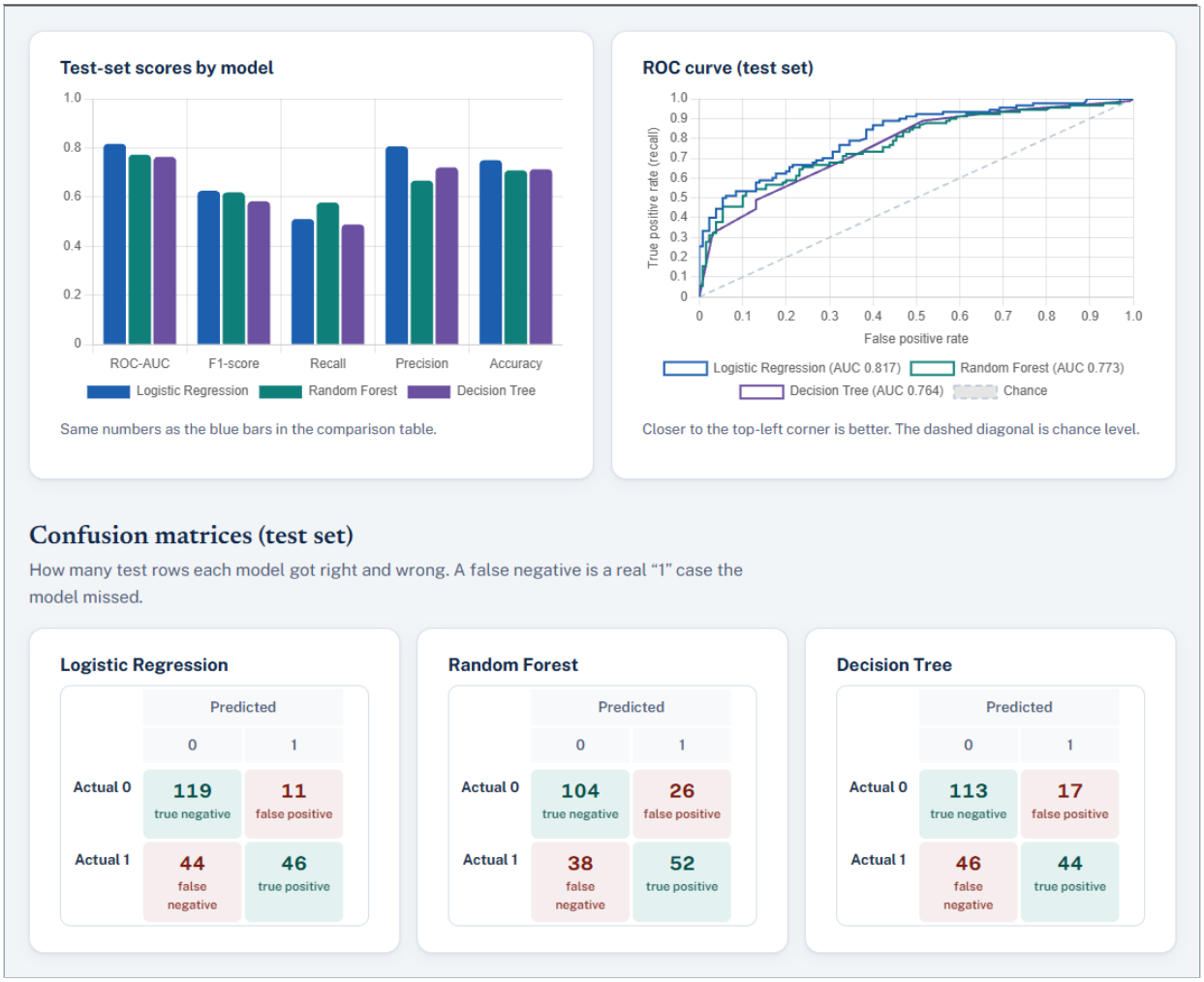


Figure 5.6: Test-Set Scores by Model, ROC Curves and Confusion Matrices (Job #5)

Figure 5.6 plots the test-set scores, the ROC curves of the three models against the chance diagonal, and their confusion matrices. The counts are listed in Table 5.6; the 220 test rows contain 130 negatives and 90 positives.

Model	TN	FP	FN	TP	Correct / 220
Logistic Regression (selected)	119	11	44	46	165
Random Forest	104	26	38	52	156
Decision Tree	113	17	46	44	157

Table 5.6: Confusion Matrix Counts on the Held-Out Test Set (Job #5)

Discussion. Logistic Regression, chosen by cross-validation, also had the best held-out ROC-AUC (0.817), accuracy (0.750) and precision (0.807): when it predicted the positive class it was right about four times out of five. Its recall, however, was only 0.511, meaning it missed 44 of the 90 positive test rows. Random Forest found more positives (recall 0.578, 52 true positives) at the cost of more than twice as many false alarms (26 against 11), and it took about twenty times longer to tune. The shallow Decision Tree (max_depth = 3) was the weakest on ROC-

AUC. The confusion-matrix counts reproduce the reported metrics exactly; for example, Logistic Regression accuracy = (119 + 46) / 220 = 0.750 and precision = 46 / (46 + 11) = 0.807. Cross-validation accuracy (0.696) is lower than test accuracy (0.750), which is within the normal variation expected from a single 220-row test set. Because the dataset is synthetic, these scores demonstrate the software workflow only.

5.8 Model Interpretation and Feature Importance

The Results page includes a "What each model relied on" panel. For Logistic Regression it shows the coefficients on the standardised inputs; for the tree models it shows impurity-based importance. In Job #5, with only two features, both models leaned mainly on Glucose. The Logistic Regression coefficients were approximately 0.51 for Glucose and 0.11 for Age (Figure 5.7), and the Random Forest importance was approximately 0.70 for Glucose and 0.30 for Age (Figure 5.8). The same pages also show the rows and classes used in the split.

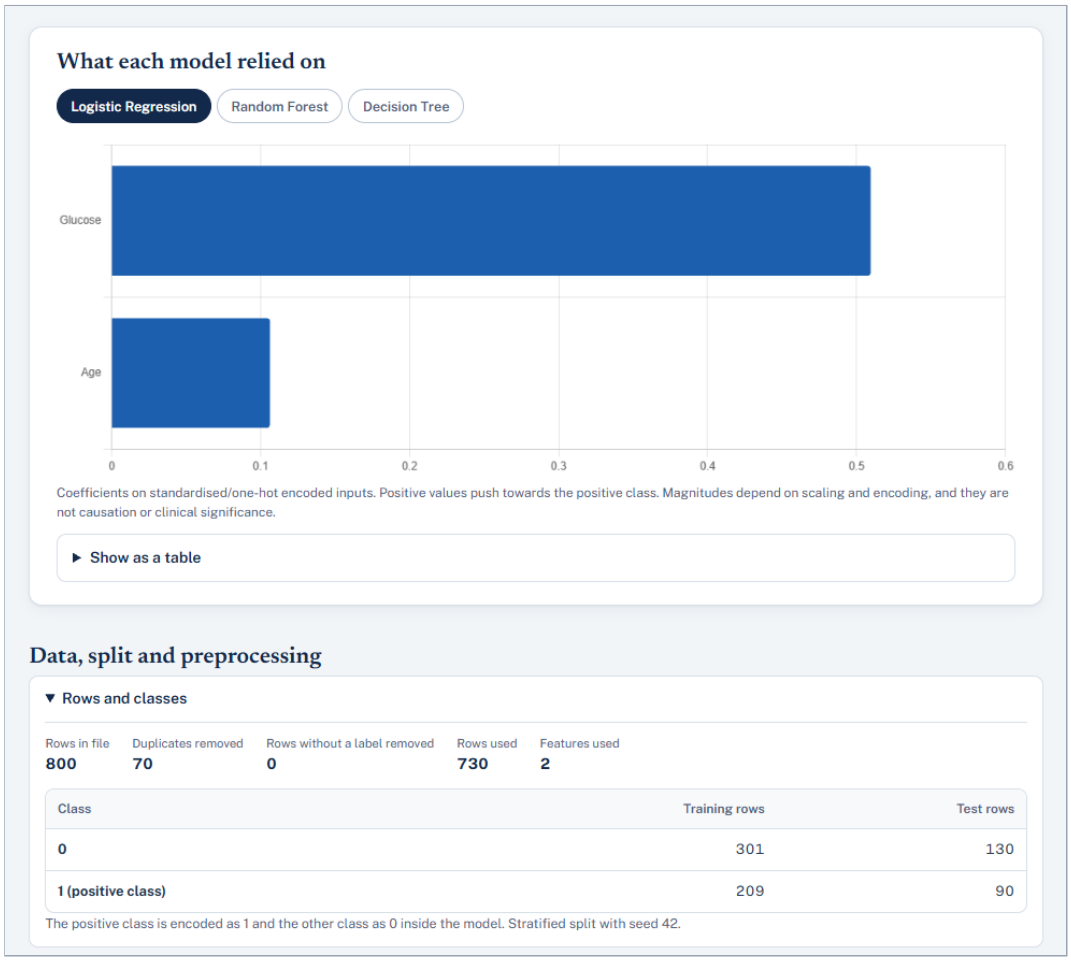


Figure 5.7: Logistic Regression Coefficients and Data Split Details (Job #5)

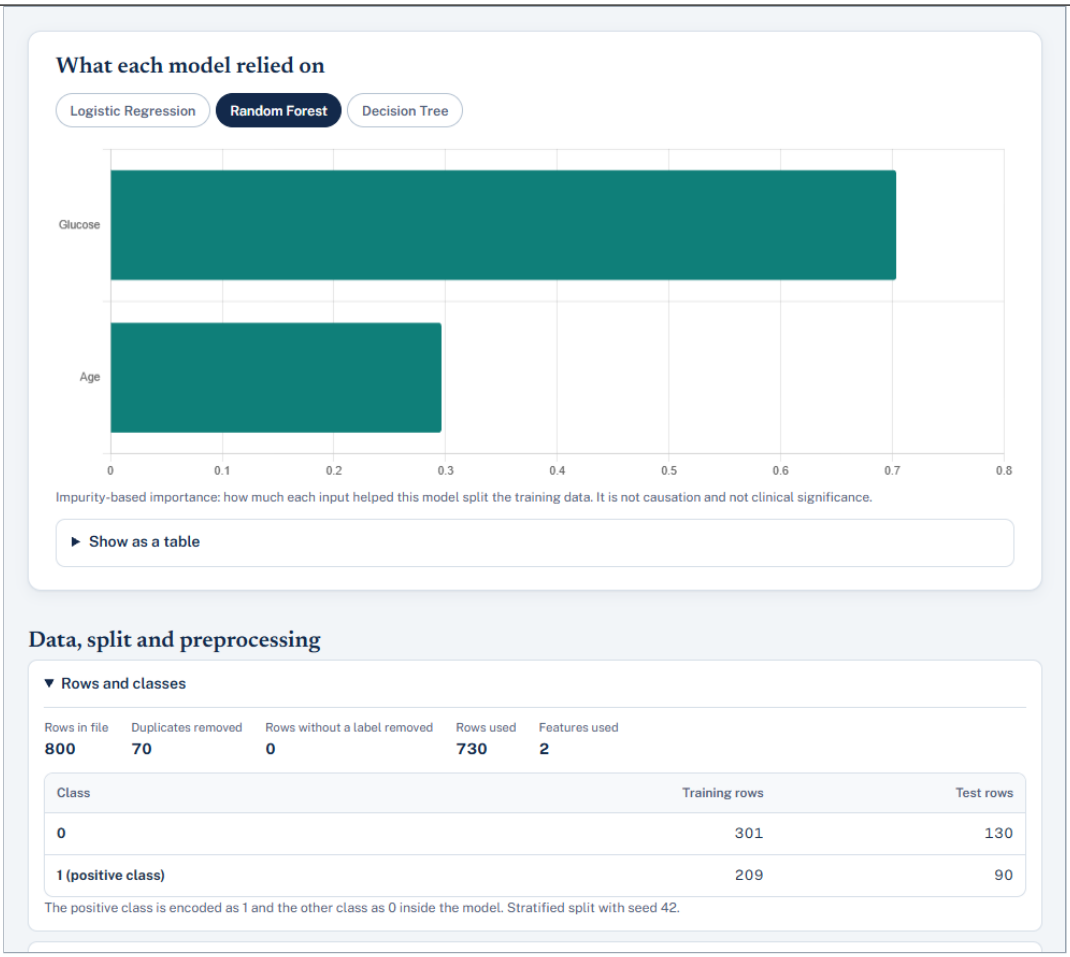


Figure 5.8: Random Forest Impurity-Based Feature Importance (Job #5)

Coefficient magnitudes depend on scaling and encoding, and impurity-based importance describes how much an input helped a model split the training data. Neither is causation or clinical significance, and the application states this beneath each chart.

5.9 Comparison with an Eight-Feature Training Run

A separate training run on the same synthetic dataset used eight model inputs: Glucose, BMI, Age, BloodPressure, DiabetesPedigreeFunction and the one-hot encoded PhysicalActivity levels (High, Moderate, Low). Its test set contained 160 rows (96 negative, 64 positive). Figure 5.9 shows its confusion matrices and Logistic Regression coefficients, and Table 5.7 lists metrics calculated from those confusion-matrix counts.

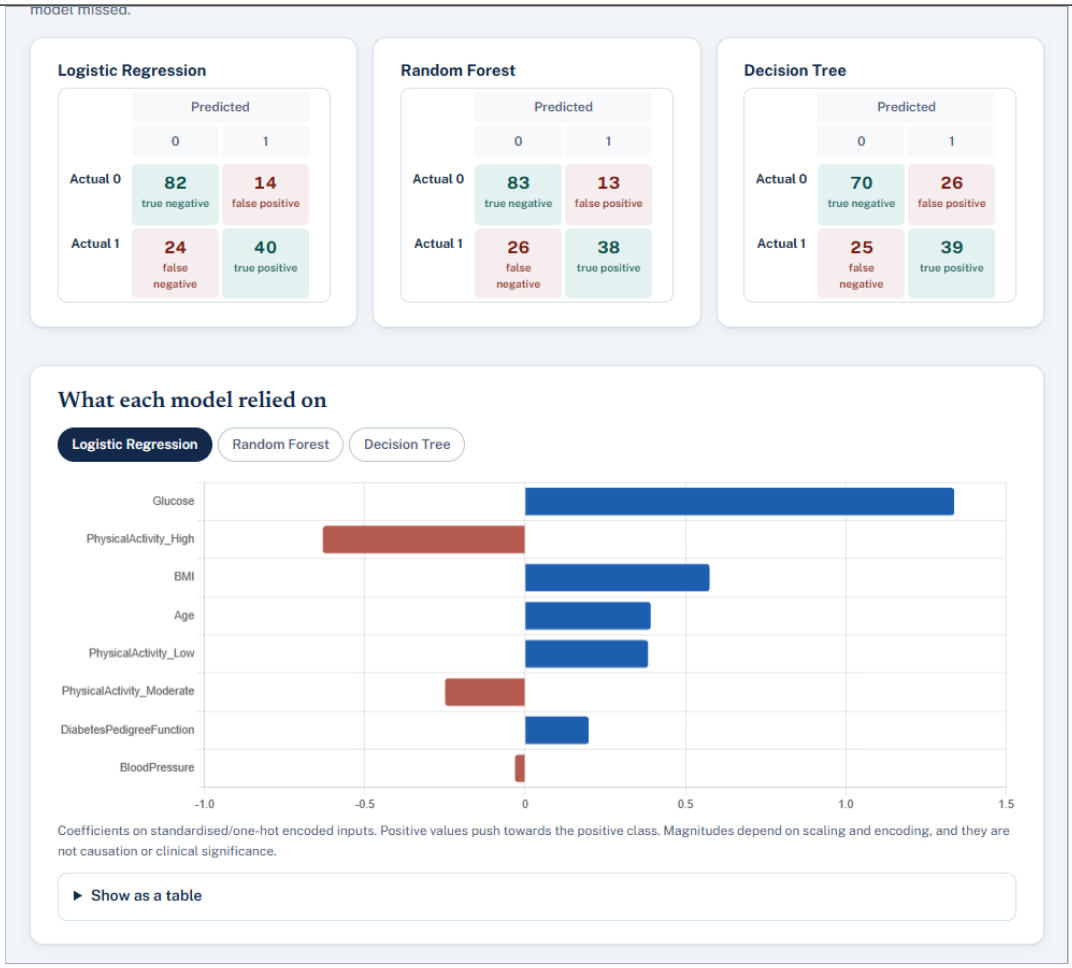


Figure 5.9: Confusion Matrices and Logistic Regression Coefficients of an Eight-Feature Training Run

Model	TN	FP	FN	TP	Accuracy	Precision	Recall
Logistic Regression	82	14	24	40	0.763	0.741	0.625
Random Forest	83	13	26	38	0.756	0.745	0.594
Decision Tree	70	26	25	39	0.681	0.600	0.609

Table 5.7: Test-Set Metrics of the Eight-Feature Run, Calculated from Its Confusion-Matrix Counts

With more inputs, Logistic Regression detected a larger share of positive rows (recall 0.625 against 0.511 in Job #5). Its coefficients show Glucose with the strongest positive weight (about 1.3), followed by BMI (about 0.57), Age (about 0.39), PhysicalActivity_Low (about 0.38) and DiabetesPedigreeFunction (about 0.20), while PhysicalActivity_High (about −0.63) and PhysicalActivity_Moderate (about −0.25) pushed predictions towards the negative class. BloodPressure was close to zero. Figure 5.10 shows that the Decision Tree of the same run used only four inputs: Glucose (about 0.75), BMI (about 0.14), DiabetesPedigreeFunction and Age. The remaining inputs had zero importance because a shallow tree uses only a few splits. These values are read from the charts and are approximate.

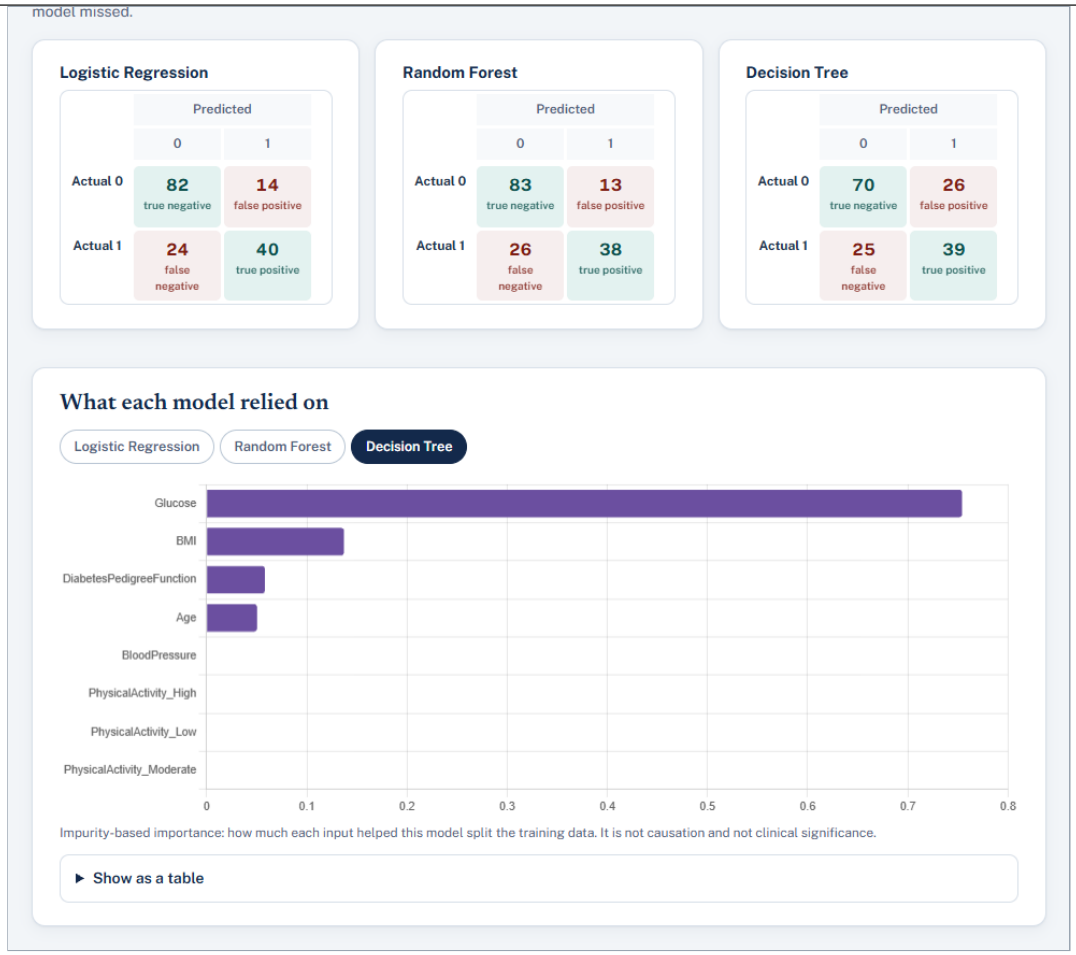


Figure 5.10: Decision Tree Feature Importance of the Eight-Feature Training Run

5.10 Summary of Recorded Training Jobs

The History page (Figure 6.4) lists every job stored in the database. All five recorded jobs used the same synthetic file and trained all three algorithms; they differ in split, selection metric and tuning level.

Job	Created	Selection metric (tuning)	Split	Status	Best CV score
#1	Oct 28, 2026, 2:41 AM	ROC-AUC (Balanced)	80/20	Completed	0.821
#2	Oct 30, 2026, 11:07 AM	Accuracy (Thorough)	75/25	Completed	0.740
#3	Oct 30, 2026, 2:06 PM	ROC-AUC (Balanced)	80/20	Completed	0.821
#4	Oct 30, 2026, 4:05 PM	Accuracy (Balanced)	80/20	Completed	0.748
#5	Oct 30, 2026, 4:11 PM	Accuracy (Balanced)	70/30	Completed	0.696

Table 5.8: Training Jobs Recorded in the Application Database

The best cross-validation scores are measured with different metrics (ROC-AUC or accuracy) and different features, so they should not be compared directly across jobs.

5.11 Test Strategy

Area	**Checks**
Data validation	Reject non-CSV/oversized files, duplicate or blank headers, invalid target, too few rows/classes, and unsupported high-cardinality text.
ML core	Missing-value handling, stratified split, CV on training data only, and prediction after reloading the saved pipeline.
REST integration	Profile data, create job, poll status, fetch results, list models, predict, and handle unavailable ML service.
Persistence	Jobs/models/history survive restart with file-based H2; avoid persisting prediction input values unnecessarily.
UI	Wizard navigation, feature selection, progress, error messages, chart rendering and model-specific prediction fields.
Security	Upload limits, safe filenames, input validation, restricted artifact loading, and no sensitive details in error messages.

Table 5.9: Test Strategy

Chapter 6
Snapshots

This chapter presents the working web application. It is served by Spring Boot on port 8080 and has five pages in its navigation bar: Dashboard, Train Model, Results, Predict and History. Every page carries the banner "Educational project. Not a medical device, and it cannot diagnose diabetes."

6.1 Dashboard

The Dashboard introduces the application and shows a System Status panel that checks the web server, the H2 database and the Python ML service, each marked Online. The panel notes that the browser talks only to the web server, which calls the ML service internally. Below it, four counters show the number of training jobs, completed jobs, saved models and predictions made (Figure 6.1; captured after the first three jobs).

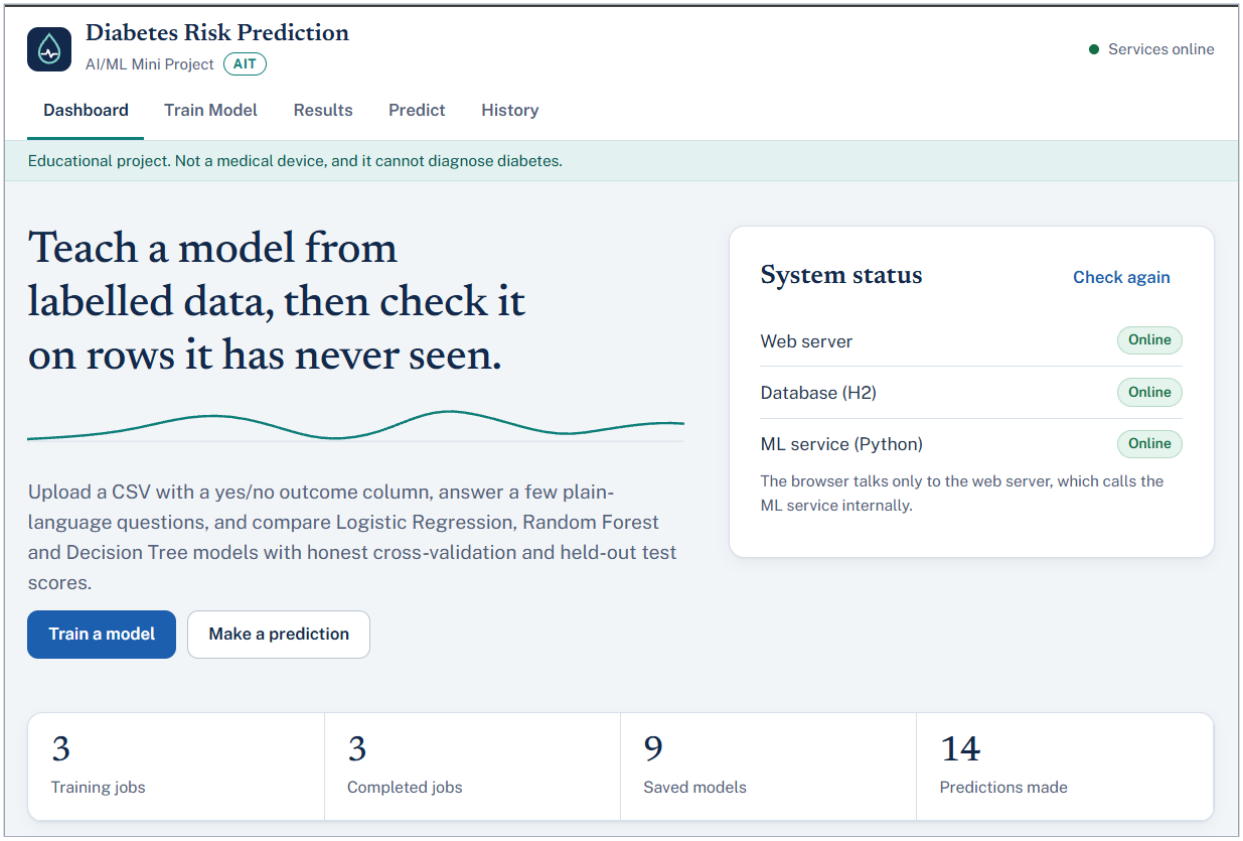


Figure 6.1: Dashboard with System Status and Usage Counters

Further down, the Dashboard lists recent training jobs with their best cross-validation score and a link to the results, and a "How it works" section explains the five steps of the workflow (Figure 6.2).

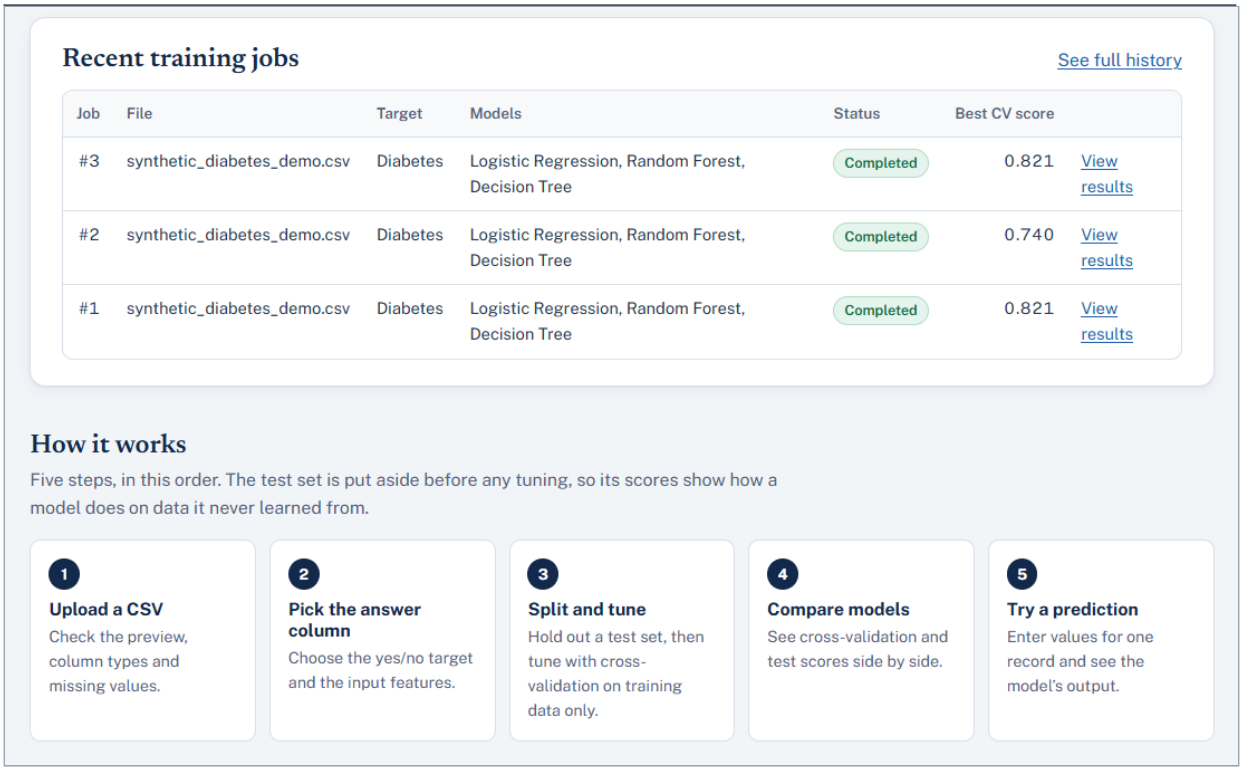


Figure 6.2: Dashboard: Recent Training Jobs and "How It Works" Steps

6.2 Results Page

The Results page has a job selector, CSV and JSON report downloads, a synthetic-data warning, the job configuration and training notes, the model chosen by cross-validation, the comparison table, test-set and ROC charts, confusion matrices, feature signals and a data-split panel. These views are shown in Chapter 5: Figure 5.4 to Figure 5.10.

6.3 Make a Prediction Page

The user selects a saved model, listed newest first. The form is built from the exact features that model was trained on; for model #13 (Logistic Regression from Job #5) these are Glucose and Age. A summary card shows the target, positive class, CV accuracy, test ROC-AUC, recall and precision, and the pipeline and scikit-learn version. Each input shows the training range and median (Glucose 55 to 200, median 118; Age 22 to 81, median 38), and values outside the range are allowed but trigger a warning. "Fill with example values" uses the training medians, and the Result panel carries an educational disclaimer (Figure 6.3).

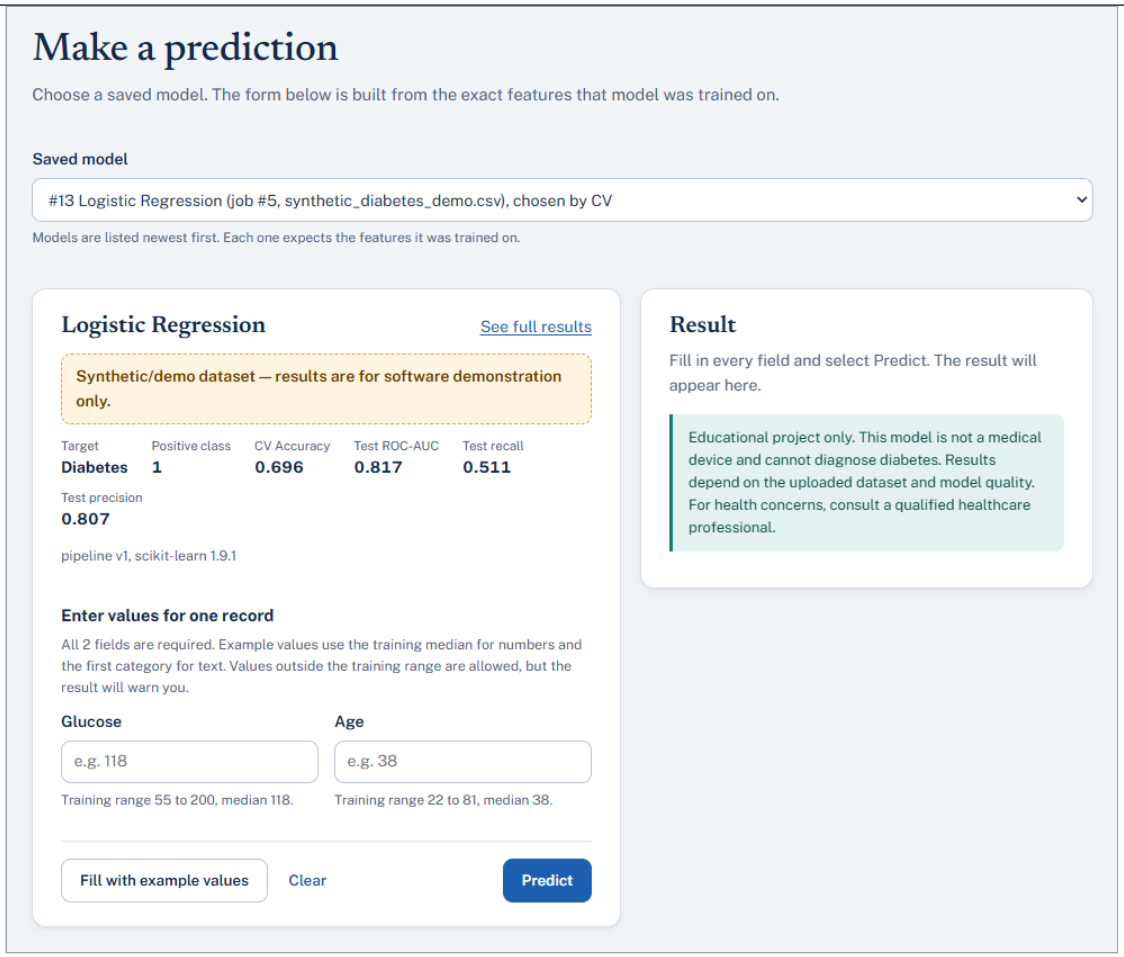


Figure 6.3: Make a Prediction Page for Saved Model #13 (Logistic Regression, Job #5)

Two numbers should be distinguished. The **model-estimated probability** applies only to the single record that was entered, whereas the **test metrics** on the model card (for example, test ROC-AUC 0.817) describe how the model performed overall on unseen test data.

6.4 History Page

The History page lists every training job with its creation time, file, target, models, selection metric, split, status and best CV score, with links to the results and the CSV report (Figure 6.4).

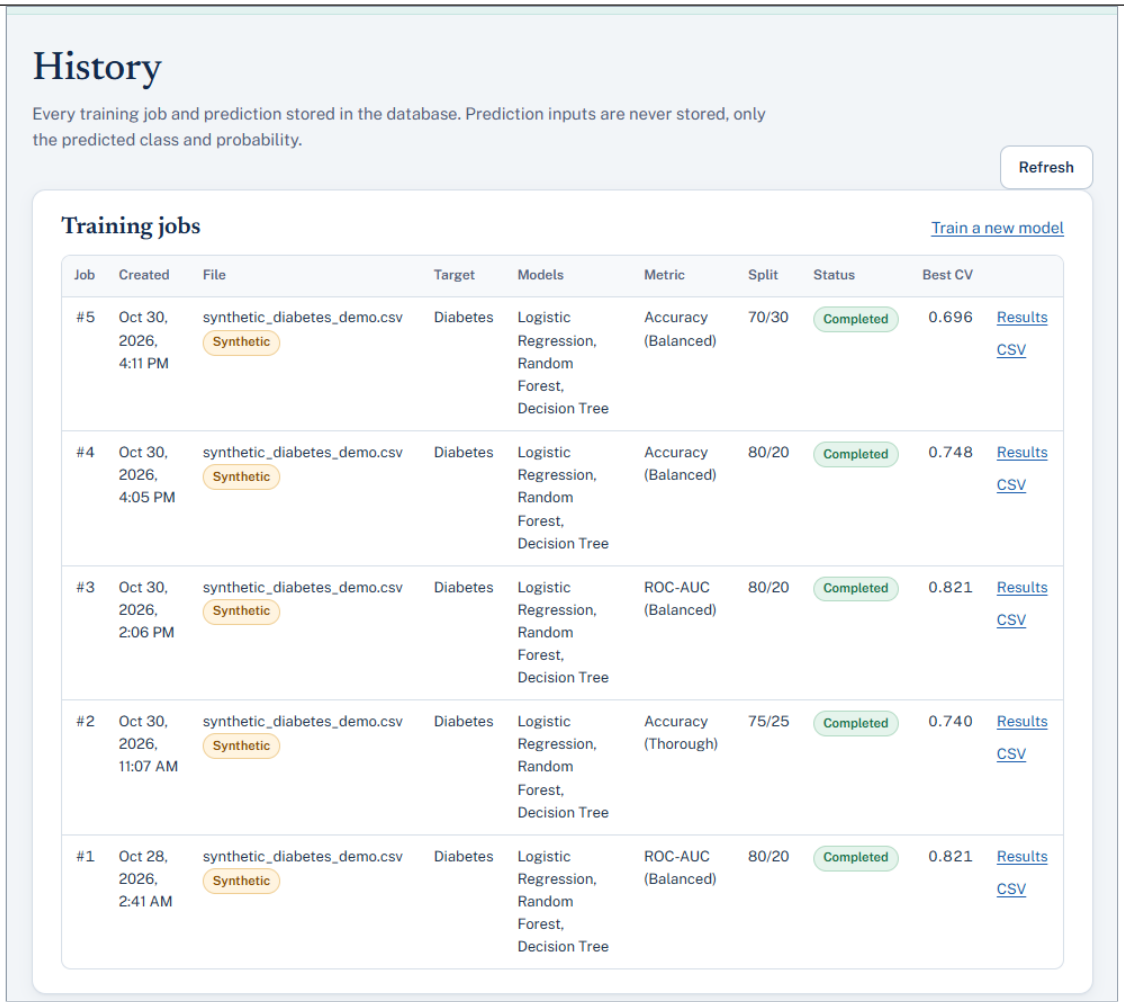
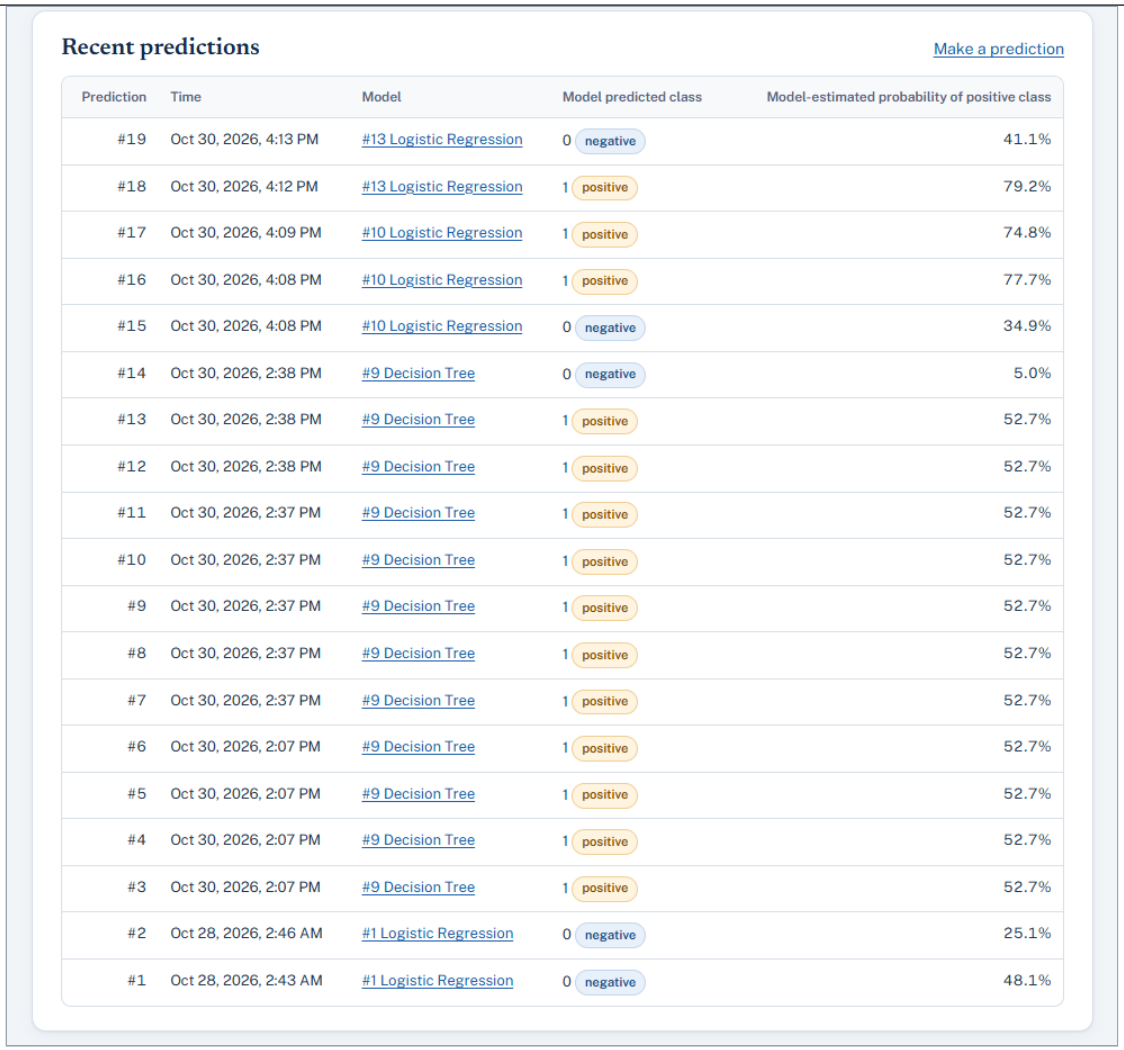


Figure 6.4: History Page: Training Jobs Stored in the Database

The same page lists recent predictions with the model used, the predicted class and the model-estimated probability of the positive class (Figure 6.5). Prediction inputs are never stored. Several predictions from model #9 (a Decision Tree) share the same probability of 52.7%, because a shallow tree assigns the same probability to every record that reaches the same leaf.



Recent predictions Make a prediction

Prediction	Time	Model	Model predicted class	Model-estimated probability of positive class
#19	Oct 30, 2026, 4:13 PM	#13 Logistic Regression	0 negative	41.1%
#18	Oct 30, 2026, 4:12 PM	#13 Logistic Regression	1 positive	79.2%
#17	Oct 30, 2026, 4:09 PM	#10 Logistic Regression	1 positive	74.8%
#16	Oct 30, 2026, 4:08 PM	#10 Logistic Regression	1 positive	77.7%
#15	Oct 30, 2026, 4:08 PM	#10 Logistic Regression	0 negative	34.9%
#14	Oct 30, 2026, 2:38 PM	#9 Decision Tree	0 negative	5.0%
#13	Oct 30, 2026, 2:38 PM	#9 Decision Tree	1 positive	52.7%
#12	Oct 30, 2026, 2:38 PM	#9 Decision Tree	1 positive	52.7%
#11	Oct 30, 2026, 2:37 PM	#9 Decision Tree	1 positive	52.7%
#10	Oct 30, 2026, 2:37 PM	#9 Decision Tree	1 positive	52.7%
#9	Oct 30, 2026, 2:37 PM	#9 Decision Tree	1 positive	52.7%
#8	Oct 30, 2026, 2:37 PM	#9 Decision Tree	1 positive	52.7%
#7	Oct 30, 2026, 2:37 PM	#9 Decision Tree	1 positive	52.7%
#6	Oct 30, 2026, 2:07 PM	#9 Decision Tree	1 positive	52.7%
#5	Oct 30, 2026, 2:07 PM	#9 Decision Tree	1 positive	52.7%
#4	Oct 30, 2026, 2:07 PM	#9 Decision Tree	1 positive	52.7%
#3	Oct 30, 2026, 2:07 PM	#9 Decision Tree	1 positive	52.7%
#2	Oct 28, 2026, 2:46 AM	#1 Logistic Regression	0 negative	25.1%
#1	Oct 28, 2026, 2:43 AM	#1 Logistic Regression	0 negative	48.1%

Figure 6.5: History Page: Recent Predictions with Predicted Class and Probability

6.5 Health Information Section and Footer

A "Did you know?" section gives short background facts about diabetes from public health agencies (the World Health Organization, the US Centers for Disease Control and Prevention and the US National Institute of Diabetes and Digestive and Kidney Diseases), each with a source link. The footer repeats that predictions are not medical advice (Figure 6.6).

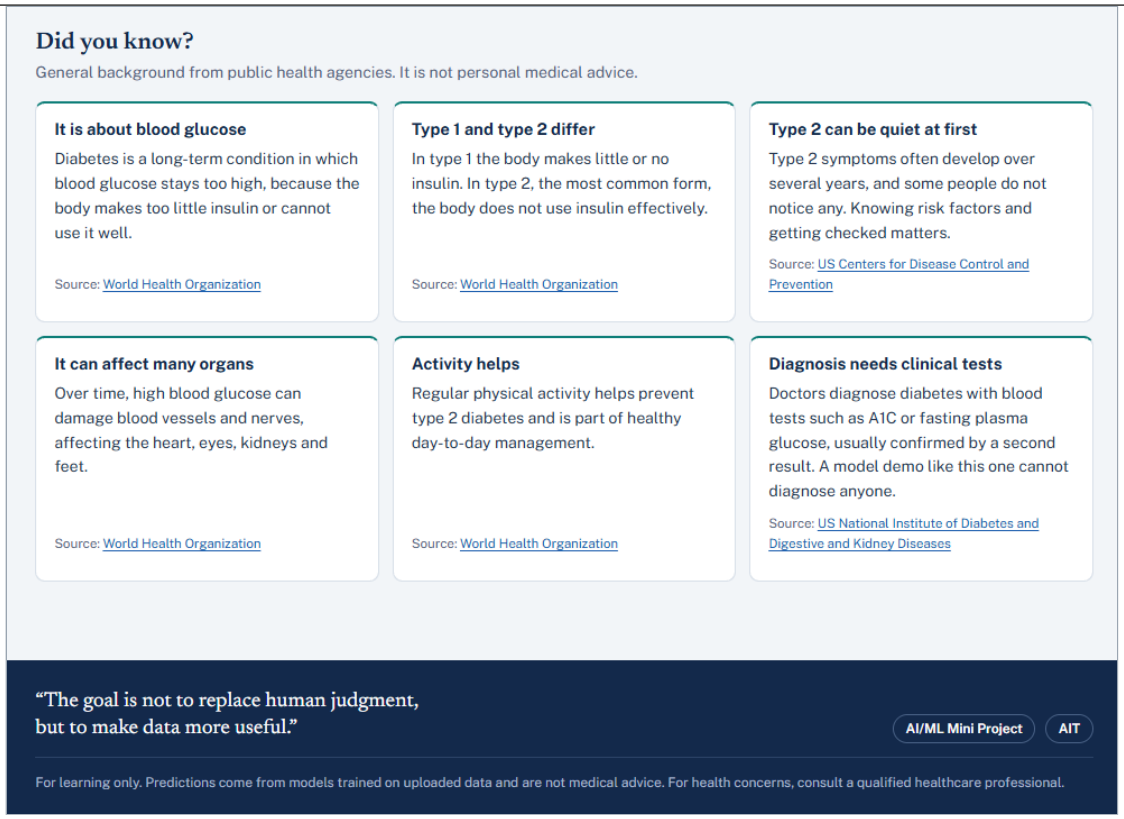


Figure 6.6: "Did You Know?" Health Information Section and Footer

Chapter 7
Conclusion and Future Scope

7.1 Conclusion

GlucoGuard demonstrates how a machine-learning experiment can be delivered as a maintainable full-stack application. The browser provides a guided workflow; Spring Boot supplies the API boundary, job management, validation, and persistence; FastAPI encapsulates data preparation, training, evaluation, artifact storage, and prediction. A complete preprocessing pipeline keeps training and inference consistent, while separate cross-validation and test scores make evaluation more transparent.

In the recorded Job #5, Logistic Regression was selected by 5-fold cross-validation (mean accuracy 0.696) and reached a held-out ROC-AUC of 0.817, accuracy of 0.750 and precision of 0.807 on 220 test rows. Its recall of 0.511 shows that it missed about half of the positive rows, and a run with eight features raised recall to 0.625. These figures illustrate why several metrics must be read together.

The central learning outcome is the end-to-end discipline of validating data, avoiding leakage, comparing models with suitable metrics, saving reproducible artifacts, handling errors, and explaining limitations honestly. Because the demonstration dataset is synthetic, the application should be presented as an educational prototype, not a diagnostic system.

7.2 Limitations and Responsible Use

- Synthetic labels may not represent real-world prevalence, demographics, measurement practices, or clinical outcomes.
- Scores on synthetic data cannot establish medical usefulness or readiness for deployment.
- Clinical deployment would require representative governed data, external validation, bias analysis, calibration, threshold review, privacy/security controls, clinician oversight, and regulatory review.
- Feature importance indicates association or model reliance, not causation.
- Data quality, missingness patterns, class imbalance, and distribution shift can change the apparent performance.
- Predictions must not be used to make personal medical decisions; users should consult qualified healthcare professionals.

7.3 Future Scope

1. Evaluate on a documented, ethically sourced representative dataset; perform external validation and subgroup analysis.

2. Add probability calibration and configurable thresholds with explicit precision–recall trade-offs.
3. Add experiment versioning, dataset fingerprints, model lineage, reproducible seeds, and comparison exports.
4. Improve security with authentication, authorisation, upload quotas, private storage, service API keys, and audit controls.
5. Add container deployment, structured monitoring, request tracing, and continuous integration tests after local workflows are verified.
6. Improve explainability using permutation importance or SHAP where appropriate; do not interpret association as causation.
7. Add automated tests for schema mismatch, unseen categories, missing values, corrupted artifacts and unavailable services.

Chapter 8
References

[1] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/

[2] Scikit-learn Developers, "Scikit-learn user guide: LogisticRegression, RandomForestClassifier, DecisionTreeClassifier, Pipeline, ColumnTransformer, GridSearchCV and metrics." [Online]. Available: https://scikit-learn.org/stable/

[3] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[4] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[5] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.

[6] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artif. Intell. (IJCAI)*, Montreal, QC, Canada, 1995, pp. 1137–1143.

[7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, 2012.

[8] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proc. Annu. Symp. Comput. Appl. Med. Care*, 1988, pp. 261–265.

[9] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[10] FastAPI, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

[11] Uvicorn, "Uvicorn documentation." [Online]. Available: https://www.uvicorn.org/

[12] VMware, "Spring Boot reference documentation." [Online]. Available: https://docs.spring.io/spring-boot/

[13] VMware, "Spring Data JPA reference documentation." [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/

[14] H2 Database Engine, "H2 database documentation." [Online]. Available: https://www.h2database.com/

[15] Pandas Development Team, "pandas documentation." [Online]. Available: https://pandas.pydata.org/docs/

[16] Chart.js Contributors, "Chart.js documentation." [Online]. Available: https://www.chartjs.org/docs/

[17] World Health Organization, "Diabetes," fact sheet. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/diabetes

[18] Centers for Disease Control and Prevention, "Diabetes." [Online]. Available: https://www.cdc.gov/diabetes/

[19] National Institute of Diabetes and Digestive and Kidney Diseases, "Diabetes tests and diagnosis." [Online]. Available: https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis

[20] American Diabetes Association Professional Practice Committee, "2. Diagnosis and classification of diabetes: Standards of Care in Diabetes—2025," *Diabetes Care*, vol. 48, suppl. 1, pp. S27–S49, 2025.

[21] GlucoGuard project source materials: README, ML training/preprocessing/evaluation/prediction modules, and Java controllers/services.